
PARNESS: A PAPER HARNESS FOR END-TO-END AUTOMATED SCIENTIFIC RESEARCH WITH DYNAMIC WORKFLOWS, FULL-TEXT INDEXING, AND CROSS-RUN KNOWLEDGE ACCUMULATION

A PREPRINT

Yuchen Wang^{*1} and Zhongzhi Luan^{†1}

¹Sino-German Joint Software Institute, Beihang University, Beijing, China

May 2026

ABSTRACT

Recent autonomous research systems — AI-Scientist v1/v2 [1, 2], PaperOrchestra [3], OpenAGS, Karpathy’s *autoresearch*, AutoSOTA [4], Tongyi DeepResearch [5], InternAgent [6] and ResearchAgent [7] — show that LLM agents can ideate, run experiments and write papers, but each fixes a particular control-flow shape (linear pipeline, hard-coded state machine, single-agent loop, or skill-pack with a fixed five-agent recipe) at the framework level. We argue that this rigidity has *five* structural roots: (1) research workflows are dynamic and discipline-specific (lab experiments, surveys, simulations, theory, ablations all loop differently and discuss differently); (2) ideation is bounded by LLM context [14] and cross-domain ideation depends on accumulated knowledge a single context cannot hold; (3) relying on paper summaries alone misses what is in the body, yet legal full-text access is uneven — so the *cumulative* corpus, not the *instant* corpus, must do the work; (4) a paper’s open-source repository (typically on GitHub) is often the only complete specification of its experimental scheme, and the paper↔code correspondence is a first-class research artefact that current systems neglect; (5) no existing tool persists cross-run knowledge in a form that can be retrieved into a finite LLM context.

We present PARNESS³, an open-source framework built on four design moves that map directly onto these five roots. (i) A thin DAG kernel with a four-field Agent contract decouples scheduling from domain semantics, so any discipline’s loop and discussion mode is expressible as user-editable YAML rather than orchestrator code. (ii) A full-text PDF-parsing and literature-library subsystem indexes paper bodies, figures and tables as typed objects, with graceful abstract-only fall-back so the system’s reading capability grows monotonically with use. (iii) A knowledge-graph index over papers, *ideas*, *experiments* and *code repositories*, with scenario-typed retrieval (similar / contradictory / cross-domain / counter-intuitive), surfaces a focused slice of the cumulative corpus into each LLM call. (iv) A deliberately small extension surface lets any modern coding agent (Claude Code, Cursor, Copilot, OpenCode, Kilo Code) add, remove or replace any module without a custom plug-in. The reference implementation runs the full research life-cycle end-to-end on real document collections; to our knowledge it is the first open-source system that combines declarative pipeline composition, full-PDF + code-repository indexing, and persistent cross-run knowledge in a single codebase.

Keywords Automated scientific research · LLM agents · DAG orchestration · Multi-agent systems · Knowledge graphs · Pipeline DSL · Full-text PDF parsing · Paper-code linking · Cross-run accumulation

^{*}First author and corresponding author. wangyuchen21@buaa.edu.cn

[†]Corresponding author. rick710055@263.net

³PARNESS stands for *paper harness*: a thin runtime that binds heterogeneous research components into a single declarative pipeline. Source code and pipeline configurations: <https://github.com/gtrhythm/PARNES>.

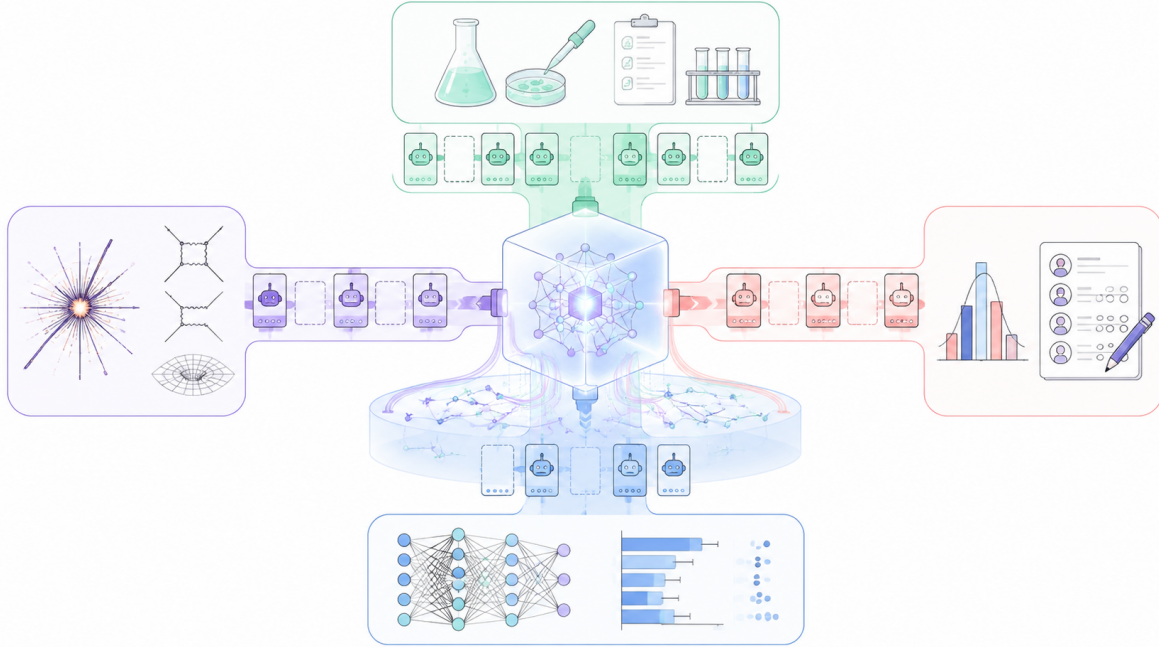


Figure 1: PARNESS as a paper harness. A single thin DAG kernel (centre) drives many different research scenarios as parallel *lanes*: wet-lab biology, social-science surveys, ML systems benchmarks, and theoretical/simulation studies. Each lane is a chain of pluggable agent modules that the user can swap, extend, or re-wire through ordinary YAML and through any GUI/TUI coding agent. A typed knowledge graph below the kernel feeds every lane. The contribution of this work is the *kernel inside the machine*: a thin runtime plus a four-field contract that lets disciplines, scenarios and discussion modes be expressed as data rather than orchestrator code, and a knowledge layer that lets every run accumulate into a long-lived corpus.

1 Introduction

The past two years have produced a wave of *autonomous research systems* that aspire to take a research question — or even a research *area* — and return reviewed scientific output without human intervention. Sakana AI’s AI-SCIENTIST v1 [1] demonstrated that GPT-4-class models [35] can drive a five-stage pipeline (idea → novelty check → experiment → writing → review) on template tasks; v2 [2] replaced the templates with a Best-First Tree Search over hypotheses and produced workshop-grade papers. Concurrently, PaperOrchestra [3] formalised a fixed five-agent writing recipe (Outline / Plotting / Lit Review / Section Writing / Refinement) packaged as host-executable skills; Karpathy’s *autoresearch* reduced neural-architecture search to a 1,000-line program-as-skill; AutoSOTA [4] optimised 105 already-published papers’ code; Tongyi DeepResearch [5] treated deep retrieval as a 30 B-A3B MoE end task; InternAgent [6] combined generation, verification and evolution agents for hypothesis discovery; and ResearchAgent [7] formulated iterative idea generation over scientific literature. A parallel literature on multi-agent collaboration [30, 31, 32, 33] provides the building blocks but does not target end-to-end research itself.

Five structural limitations. Read together — with the necessary acknowledgement that no single system has *all* of these in its strongest form — these systems share five structural problems that motivate the present work.

(L1) Frozen, monoculture control flow. The shape of the research pipeline — how many stages, what runs in parallel, when to loop, what form the discussion takes — is hard-coded in Python control flow or in a `WorkflowState` enum. Adding an extra review pass, swapping an LLM call, or expressing a discipline whose loop is not “ideate → experiment → write” requires modifying the orchestrator. The research *process* varies enormously across disciplines and questions: a wet-lab biology study iterates on protocols and replicates; a social-science investigation cycles through survey design, IRB, recruitment, coding; a CS systems paper runs benchmarks; a theoretical physics paper proves; a meta-analysis aggregates. Discussion modes also differ: idea-only multi-round critique before any experiment; experiment-result-driven idea refinement; paper-result-driven post-hoc discussion. Today’s autonomous research systems target a single shape — typically the small-scale ML benchmark loop — and provide no obvious way to express the others.

(L2) Bounded ideation, especially across domains. LLM ideation is fundamentally limited by the model’s context window. Even the longest-context frontier models cannot ingest, in one prompt, the cumulative literature a human

researcher carries to a cross-domain question; and what they can ingest exhibits the well-documented *lost-in-the-middle* phenomenon [14]. Single-call ideators therefore see only a subsample of what would inform a real research move, and the subsample is chosen poorly relative to the long-tail accumulated knowledge that drives cross-domain creativity [15]. Researchers attempting cross-domain work without prior accumulation in one of the domains often cannot even formulate a starting direction.

(L3) Episodic, non-indexed full-text reading. Several existing systems do load full PDFs at the moment of use: AI-SCIENTIST v1/v2 [1, 2] parse paper bodies via `pymupdf` during novelty checking, and Tongyi DeepResearch [5] reads full text in its information-seeking loop. The problem is not whether the body is read *once*, but that the parsed text is not retained as a queryable, typed corpus that subsequent steps and subsequent runs can search. In the systems above, a paper is parsed for the current call and then forgotten — its detailed content, figure structure, and table values are not added to a long-lived index. PaperOrchestra [3], in contrast, never reads paper bodies at all; its literature agent uses the Semantic Scholar API which exposes only title and abstract for the discovery step. In both regimes the system has nowhere to compensate when full text is unavailable for a given paper, because nothing was kept from previous reads. The right model is the researcher with deep prior accumulation, who understands a new paper from its abstract because the abstract resolves *against* their accumulated context.

(L4) Code access exists, but the paper↔code link is per-task, not corpus-level. Several systems do touch experimental code: AutoSOTA [4] explicitly optimises 105 already-published papers’ code-bases through an Idea-Library hypothesis tracker; AI-Scientist [1, 2] writes training scripts through Aider; Karpathy’s *autoresearch* mutates a single `train.py` under a coding-agent budget. What none of them builds is a *typed, queryable graph* that links paper concepts to specific repository fragments and across-paper to similar repositories from neighbouring research lines. As a result the systems cannot answer corpus-level questions like “find me the preprocessing pattern this paper describes, but as it appears in a sibling paper that did ship code”, or “which repositories from *related* research lines implement an experimental scheme close to this newly-generated idea?”. Across CS, ML, robotics and increasingly biology, the GitHub repository is often the only complete specification of an experimental method — exact hyperparameters, preprocessing, hidden constants. The paper↔code correspondence carries information that is in neither the paper alone nor the code alone, and a corpus-level index of that correspondence serves two roles: (a) a verifier and analyser of a paper’s claims; (b) a reservoir of inspiration for related research whose experimental schemes are not openly released, including for novel ideas the system itself proposes.

(L5) No retrievable cross-run accumulation. A human researcher’s productivity depends critically on the corpus of seeds, failed leads, half-finished hypotheses, replication notes, and code links accumulated across prior projects. Existing autonomous systems treat each run as an isolated session: the crawler re-fetches papers, the ideator re-generates seeds, the reviewer re-derives standards. Yet *too much* accumulated context overflows the model. Whether one calls the LLM once per pipeline or many times per stage, the engineering challenge is the same: *how to surface the most useful slice of accumulated knowledge into a finite context window at every step*. This is the central open problem the rest of PARNES is structured to attack.

To these five we attach two further structural observations that fall out of (L1)–(L5) but are worth naming separately.

(L1+) Agent– framework conflation: domain logic (“did this idea pass the bar?”, “stop iterating when novelty plateaus”) lives inside the orchestrator code, so the *framework* knows about scoring, novelty, and termination semantics it has no business knowing. **(L1++) Coverage gaps:** most systems address a slice of the life-cycle (only ideation; only optimisation; only retrieval; only the writing of a draft). PaperOrchestra explicitly assumes a complete (*idea, experimental log, template, guidelines*) tuple as input, deferring everything upstream of writing to “the host agent”. AI-Scientist covers more stages but on hard-coded ML templates.

This work. PARNES attacks all five (and the two corollaries) by inverting the relationship between the *framework* and the *agents*. The framework is a thin DAG runtime: ~600 lines of `GraphRunner` that does topological scheduling, input/output mapping, and process-pool execution. Every domain decision — whether an idea continues to the next round, whether to fan out to parallel reviewers, when to terminate iterative refinement — is encoded by a module returning four reserved fields (§6.2). On top of this kernel we ship 130+ registered modules organised into 50 YAML pipelines covering the full life-cycle: research/crawler, full-PDF parser, code-link extractor, ideation (six cognitive roles + twelve specialty agents), experiment runner/verifier CLI, writing/review, and a Knowledge-Graph subsystem (17 adapters, 4 pipelines, 52 passing tests) that indexes papers, ideas, experiments *and code repositories* as typed graph nodes. The framework deliberately exposes its modules as drop-in slots that any GUI/TUI coding agent (Claude Code, Cursor, Copilot, OpenCode, Kilo Code) can rewrite, replace, or compose into a custom pipeline.

Contributions. We make four contributions, mapped one-to-one with the limitations above and elaborated as the four pillars of §5:

1. **(M→L1) A highly-customisable DAG kernel and a declarative pipeline DSL.** A minimal four-field Agent contract (§6.2) plus a YAML topology language with layered validation (§6.3), so disciplines and discussion modes are expressed as data, not code. Compared to programmatic orchestrators (AutoGen [31], MetaGPT [30], DSPy [34]) and skill-pack approaches (PaperOrchestra [3]), PARNESS treats the *topology* of the workflow as user-editable data and enforces correctness with a layered pipeline validator (§6.10).
2. **(M→L3) A complete full-text PDF-parsing and literature-library subsystem.** An integration of the third-party PDF-Extract-Kit [27] (used unmodified except for stability and adapter glue) feeds a typed indexing layer (§5.2); the system gracefully degrades to abstract-only when full text is unavailable, and progressively gains “abstract-comprehension” capability as the cumulative corpus grows.
3. **(M→L2,L4,L5) A KG-based knowledge index over papers, ideas, experiments and code repositories** (§6.7, §5.3). Eight-phase Neo4J indexer with four edge types — similarity, contradiction, cross-domain, derivation — so each LLM step retrieves the slice that fits its scenario. Six cognitive-role idea agents (§6.6) consume that slice in parallel. Five SQLite stores keep cross-run state durable so the next run starts where the last left off.
4. **(M→flexibility) A GUI/TUI-first extension surface.** Every module is a single class behind a single contract, registered with a one-line decorator, schema-validated against its YAML node, and made discoverable to external coding agents (Claude Code, Cursor, Copilot, OpenCode, Kilo Code) through a stable layout (§5.4). Users add, remove or replace modules and edit pipelines from inside their preferred editor.

The reference implementation is open-source: 130+ modules, 50 pipelines, 5 SQLite stores, a Neo4J KG, with broad development-test coverage; it runs end-to-end in single-node and multi-GPU configurations (§8). Source and pipeline configurations are released at <https://github.com/gtrhythm/PARNESS>.

2 Related Work

We compare PARNESS with eight recent systems that operate at adjacent points in the design space (Table 1). Three axes matter for the comparison: (i) *coverage* of the research life-cycle; (ii) *composition model* for stages and decisions; and (iii) *persistence* of cross-run knowledge.

End-to-end pipelines. AI-SCIENTIST v1 [1] fixes a five-stage pipeline (idea, novelty, experiment, write, review) in ~2,100 LoC across five Python files, locked to per-domain templates (e.g. nanoGPT, 2D-Diffusion). v2 [2] drops the template requirement by introducing a four-phase Best-First Tree Search with VLM-in-the-loop review, but still hard-codes the four phases. OpenAGS proposes “folder = agent” and a DIRECTIVE/STATUS protocol over SOUL.md role definitions, but at v0.0.4 ships placeholders for paper writing and Rust-CLI bridges.

Skill-pack approaches. PAPERORCHESTRA [3] is the most recent and closest non-PARNESS alternative. Its design is deliberately minimal: a five-agent recipe (Outline → Plotting || Lit-Review → Section-Writing → Refinement) packaged as *skill-pack instructions* that any host coding agent (Claude Code, Cursor, Antigravity, Cline) reads and executes using its native LLM, web search, and shell tools. The repo ships zero embedded API clients and zero LLM dependencies; deterministic helpers enforce citation gates, Levenshtein matching, and halt rules. Strengths: trivially pluggable across hosts, faithful to the published prompts, no key management. Weaknesses for our purposes: (i) the topology is fixed at five steps with parallelism only between Plotting and Lit-Review; (ii) the entire pipeline assumes the inputs (I, E, T, G, F) already exist (idea, experimental log, template, guidelines, optional figures), so the upstream stages of the research life-cycle (literature acquisition, hypothesis generation, experiment design *and* execution) are explicitly out of scope or delegated to a separate optional `agent-research-aggregator` skill; (iii) cross-run knowledge is not modelled — the `provenance.json` file captures input/output hashes for one run, not a corpus that the next run can build on; (iv) ideation is a single LLM call inside the host agent’s window, so context limitations (L3) hit at full strength. PARNESS and PaperOrchestra are complementary in spirit — one could plug PaperOrchestra’s writing sub-pipeline as a single PARNESS module — but their architectural commitments are different.

Single-stage specialists. Karpathy’s *autoresearch* is a ~1,000-LOC loop that runs an external coding agent under a 5-minute budget to mutate a `train.py`; it has no LLM, no literature, no paper output. AutoSOTA [4] operates at the *post-publication* boundary: 105 already-published papers’ code-bases are optimised with an Idea-Library hypothesis tracker; its core engine is closed-source. DeepResearch [5] is a 30.5 B parameter (3.3 B active) MoE retrieval agent — single-turn information seeking, no experiment, no writing.

Generation-verification systems. INTERNAGENT 1.5 [6] ships eight functional agents (Survey, Scholar, Generation, Reflection, Evolution, MethodDevelopment, Refinement, Ranking) coordinated by an 11-state `WorkflowState` enum.

Table 1: Comparison of PARNESS with eight recent autonomous research systems across nine axes. Numbers are module / pipeline / test counts where reported by the system or measured from its source. PAPER-O. = PaperOrchestra; INTERN. = InternAgent; AUTO-R. = Karpathy’s autoresearch.

Axis	v1	v2	OpenAGS	PaperO.	auto-r.	AutoSOTA	DeepRes.	Intern.	PARNESS
Composition	linear	4-stage	SOUL.md	5-step recipe	program.md	—	ReAct	11-state	DAG/YAML
Pipelines	1	1	1	1	1	0	1	1	50
Modules	5 files	~15	8 dirs	7 skills	3 files	—	—	8 cls.	130+
LLM providers	multi-if	6+	4 CLI	host LLM	external	1	1	3	6 factory
Literature srcs.	1	1	2	web+S2	0	0	1	3	18
Persistence	API only	.bib cache	FS	provenance.json	git	—	jsonl	FS	5 SQLite + KG
Knowledge graph	—	—	—	—	—	—	—	—	17 adapters
Paper writing	template	2 templ.	stub	strong	—	—	—	—	11 modules
Tests	few	—	4 files	helpers	0	—	—	—	553

ResearchAgent [7] formulates research idea generation as iterative refinement over scientific literature. Strong on hypothesis evolution, but the *paper writing* stage is absent or thin.

General multi-agent and pipeline frameworks. AutoGen [31], MetaGPT [30], ChatDev/Reflexion [32] provide multi-agent conversation as the primary abstraction; DSPy [34] compiles declarative LLM *programs*; LangChain composes prompt chains. None target the research life-cycle directly. PARNESS can wrap any of them as a single module under a four-field contract; we use AutoGen and DSPy idioms internally for some adapters but neither is a dependency.

Where PARNESS sits. PARNESS is closest in spirit to AI-SCIENTIST (full life-cycle, open-source) and PAPER-ORCHESTRA (declarative skill-style composition) but differs structurally on *four* axes simultaneously: (i) the pipeline shape is data, not code or a fixed recipe; (ii) routing decisions live in agents, not the runner; (iii) knowledge persists across runs in SQLite and Neo4J, not just per-run files; (iv) the runtime supports multi-discipline workflow shapes (lab-loop, survey, simulation, ablation, meta-analysis) by composition, not by forking. To our knowledge no prior open-source system simultaneously satisfies all four.

3 Motivation: Why a Harness, Not a Pipeline

This section grounds the abstract limitations (L1)–(L5) of §1 in five concrete observations from running prior systems and our own predecessors at scale. Each observation is mirrored by a design move in PARNESS, summarised at the end of each subsection and elaborated in the Method section (§5).

3.1 Research workflows are dynamic and discipline-specific

A research *process* is much more than “ideate, experiment, write”. A wet-lab biology study cycles through hypothesis, protocol design, IRB or biosafety review, multiple replicates, statistical re-design, then writing. A social-science investigation cycles through instrument design, ethics approval, recruitment, coding, qualitative analysis, then writing. A computer-simulation study iterates over parameter sweeps and convergence diagnostics; a theoretical-physics paper iterates over proofs; a meta-analysis iterates over inclusion criteria. Each of these has *its own* loop topology. Equally important, each has *its own discussion mode*: some research lines critique the idea in multiple rounds before any experiment runs; others refine the idea *after* each experimental result lands; others run a paper-level post-hoc discussion that drives the next paper rather than the current one. The recent literature acknowledges this implicitly by spawning per-discipline frameworks (Robin [8] and DORA AI Scientist [9] for biology, EXHYTE [10] and active-inference variants [11] for general scientific discovery, BioDisco for biology, GeoColab for geospatial code), but the per-discipline framework is the wrong unit of sharing: every discipline re-implements crawling, persistence, and review from scratch.

Design move (M1). PARNESS expresses the workflow itself as a YAML DAG over a four-field contract (Method §5.1). The same kernel runs all of: a CS-systems benchmark loop, an HEP-Lat literature-driven idea pipeline, a knowledge-graph ingestion run, and a paper-writing-only pipeline. Adding a new discipline or a new discussion mode is a YAML edit plus zero or more new modules registered through the four-field contract.

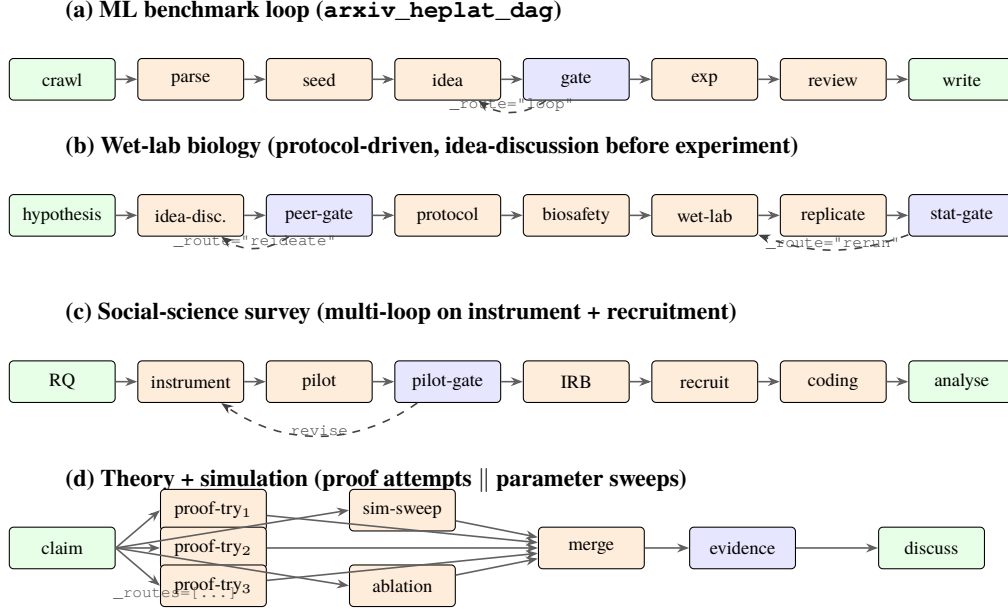


Figure 2: Four concrete PARNESS pipelines for four disciplines, all expressed in the same YAML DSL on the same DAG kernel. (a) An ML benchmark loop iterates idea generation against a quality gate, looping back to ideation if the gate score is too low. (b) A wet-lab biology pipeline holds an idea-discussion round *before* any experiment (peer-gate), and a statistical gate after replication that can trigger reruns. (c) A survey pipeline alternates instrument-pilot-revise loops, then sequential IRB / recruit / coding / analyse stages. (d) A theory pipeline fans out parallel proof attempts and parameter sweeps with `_routes`, then merges into an evidence gate. Orange = LLM-driven agent; green = I/O / persistence; blue = score-gated controller; dashed = back-edge from a routing decision. None of these shapes can be expressed in PaperOrchestra’s five-step recipe [3] or InternAgent’s eleven-state machine [6] without forking the framework.

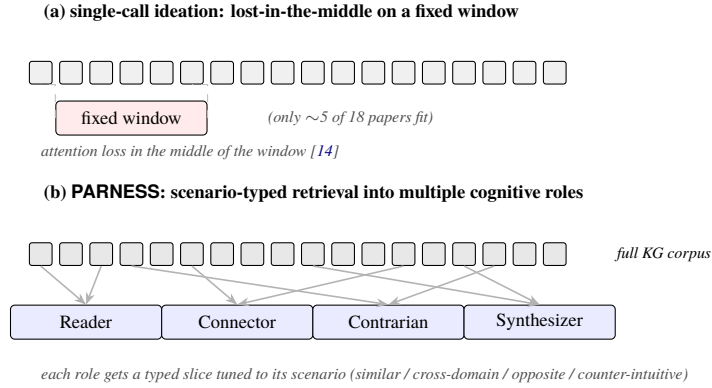


Figure 3: Cross-domain ideation under finite LLM context. (a) A single ideator with a fixed context window can only accommodate a small subsample of the corpus, and even that subsample is read with the well-documented attention bias of lost-in-the-middle [14]. (b) PARNESS separates retrieval from reasoning: the KG indexer (§5.3) holds the full corpus; each cognitive-role agent (§6.6) is wired to a *scenario-typed* retrieval (similar / cross-domain / opposite / counter-intuitive) so the same total token budget is spent on maximally different slices.

3.2 LLM ideation is bounded by context, not by talent

A human researcher rarely generates an idea from scratch in one sitting. The good ideas reflect months or years of *accumulated reading and experiments*, often across disciplines. Reproducing this in an LLM is hard for two stacked reasons. First, the long-context literature has identified a *lost-in-the-middle* pattern [14]: even when a frontier model technically supports a million-token window (e.g. Claude Opus 4.7 with its 1 M-token mode, DeepSeek-V4 in its 1 M-context build, Gemini-2.x Pro at 2 M tokens) or a quarter-million window (e.g. Claude Sonnet 4.6, GPT-class long-context models), its attention to evidence in the middle of the prompt is materially worse than to the ends. The smaller open models that researchers actually fine-tune and self-host (Llama-3 / Mistral / Qwen variants in the 32–128 K

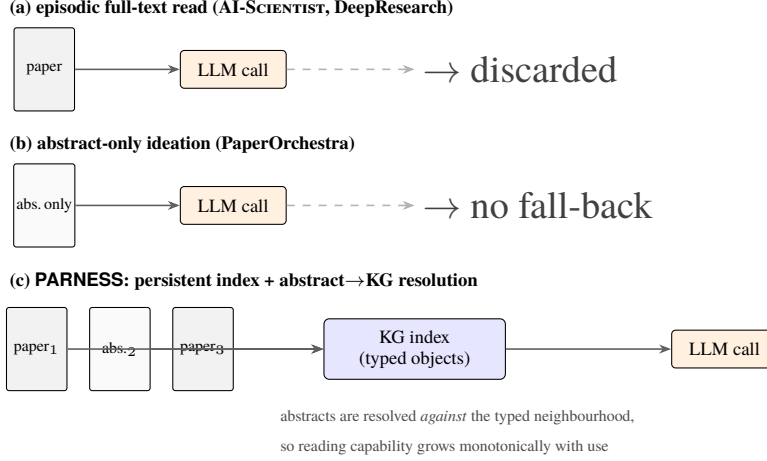


Figure 4: Reading a paper once is not the same as having it indexed. **(a)** AI-SCIENTIST [1, 2] and DeepResearch [5] do parse full PDFs, but only *episodically*: the parsed text is consumed by the current call and then discarded. **(b)** PaperOrchestra [3], by contrast, only ever sees abstracts. In both regimes there is no long-lived corpus that the next step or the next run can search. **(c)** PARNES indexes every parsed body as typed objects (layout, formulae, OCR, tables) into the KG; abstract-only inputs are resolved against the typed neighbourhood, so the system’s reading capability grows monotonically with use.

range) hit the same ceiling earlier and harder. Second, even if attention were uniform, the cumulative corpus a senior cross-domain researcher carries (textbooks, lab notes, half-finished papers, conference notes) does not fit in any of these windows. The result, in practice, is that single-call ideators in autonomous systems generate ideas that are local to whatever literature happened to be retrieved in the last query — a small, biased slice. The recent *IdeaSynth* [12] and *Many-Heads* [13] work formalises this as a multi-agent compositional problem.

Design move (M2). PARNES *splits* ideation into six cognitive roles (Reader/Analyst/Connector/Contrarian/Synthesizer/Critic; §6.6) that each see a focused slice of the corpus retrieved through the KG and SQLite stores. The Connector role specifically targets cross-domain analogies; the Contrarian targets failure modes; the Synthesizer recombines. The slice is selected by the KG retrieval adapters (§5.3), which see the typed graph rather than the raw context window. Empirically the parallel ensemble produces wider seed coverage than a single LLM at the same total token budget; we have not yet measured this rigorously and flag it as future work.

3.3 Summary-only views miss the paper; cumulative reading rescues them

A paper’s abstract is a compression of 30–50× its body. The contribution often hides in details that simply do not appear in the abstract: the exact preprocessing step that made an architecture generalise, the held-out split that exposed the bug, the table footnote that says “*three runs, mean reported*”. Several existing systems — AI-Scientist [1, 2], DeepResearch [5] — do load the full PDF when they need it, typically through a `pymupdf`-style parser at the moment of use. The problem is that the parsed text is not retained as a queryable, typed corpus. Once the current LLM call is done, the body is forgotten, so neither a later step in the same pipeline nor a later run can search it. Other systems — PaperOrchestra [3] most prominently — deliberately stop at the abstract because the literature agent runs against the Semantic Scholar API which only exposes title and abstract for the candidate-discovery step. In both regimes the system has nowhere to compensate when full text is unavailable, because nothing was kept from previous reads. The correct response is therefore *both* (i) ingest the full body when it is legally and practically available, and (ii) make every parsed body a permanent indexed asset, so subsequent abstract-only inputs can be resolved against the typed neighbourhood. A senior researcher in a field reads an abstract and, because of years of accumulated context, knows roughly what the body must say. We want PARNES to behave the same way: a fresh installation degrades gracefully on abstract-only input, but a PARNES that has indexed thousands of related papers fills in the gaps through retrieval rather than re-reading.

Design move (M3). PARNES integrates the third-party PDF-Extract-Kit [27] together with MinerU [26] as the upstream parsing engines (layout, formula, OCR, tables; §5.2) and adds the surrounding plumbing to expose the parsed objects as typed nodes to the KG indexer; the parsers themselves are not our contribution and are used essentially unmodified. When full text is missing, the indexer falls back to abstract-only embedding and relies on the cross-edges built from previously-indexed full-text neighbours. The result is a system whose reading capability *grows monotonically* with use.

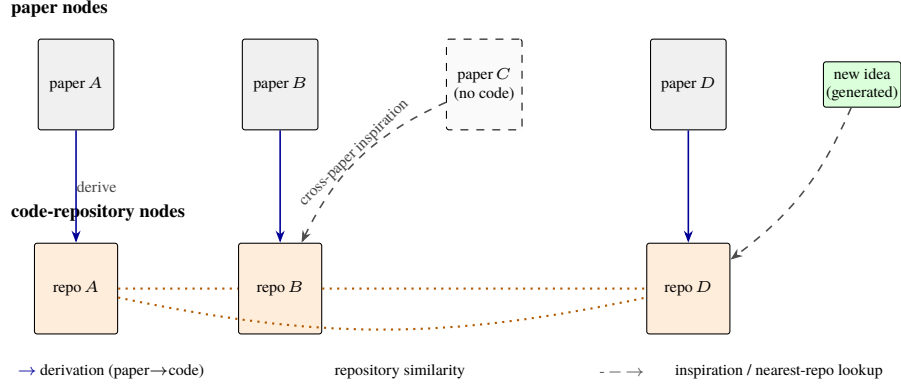


Figure 5: The PARNESS paper↔code graph. Every parsed paper that ships a repository emits a typed *derivation* edge to its code-node; repository nodes are linked to each other by similarity edges. A paper without code (paper *C*) and a freshly-generated idea both reach the closest sibling repository through cross-paper inspiration edges. Several existing systems touch code per-task (AutoSOTA [4], AI-Scientist [1, 2], autoresearch), but none builds a corpus-level typed graph of this shape.

3.4 Open-source code is the missing ground truth

In CS and ML the open-source repository is increasingly the *contract* between author and reader. Several autonomous-research systems already work with code: AutoSOTA [4] optimises 105 already-published papers’ code-bases; Paper2Code [23] generates code from papers; AI-Scientist drives an external coding agent (Aider) to write training scripts. The broader reproducibility literature [22, 24] repeatedly finds that paper-level descriptions disagree with the code that produced the headline numbers. What is missing is not code *access*, but a typed corpus-level *graph*: in every system above, the paper↔code link is implicit and per-task. A cross-paper retrieval like “find me how this paper’s preprocessing is implemented in a sibling paper that *did* ship code” is not expressible. The paper↔code link therefore does double duty: (a) it lets the system audit the paper against the code, surfacing disagreements that the abstract hides; and (b) when a related paper does *not* ship code (common for adjacent subfields), the link to a sibling paper that *does* ship code becomes a strong source of experimental inspiration. New ideas the system itself produces can also be indexed against this corpus to find the closest reference implementation.

Design move (M4). PARNESS ships a code-link extractor that parses paper bodies and abstracts for repository URLs, clones repositories under a sandbox budget, runs a code-analyser agent to type the repository contents (training entry-point, preprocessing, config, model definition), and writes typed nodes into the KG with *derivation* edges to their source paper. Cross-paper retrieval adapters (§5.3) then surface code from related papers when the LLM step is reasoning about a paper without code, or about a newly-generated idea.

3.5 Knowledge must accumulate, but accumulation must be retrievable

Researcher productivity depends on what was learned in earlier runs: which literature did not pan out, which hypotheses turned out inconsistent with experimental data, which papers from neighbouring fields turned out useful, which code patterns reliably reproduced results. Existing autonomous systems start each run from zero; the result is that running the system N times is no more informative than running it once N times in parallel. *But* simply keeping all prior context is also wrong, because of (L2)–(L4): you cannot re-stuff a year of accumulated runs into one prompt, even with a million-token window. The challenge is not single-call vs. multi-call — it is identical for both: *how do we surface the smallest, highest-value slice of the cumulative corpus into a finite context window at every step?*

The right structure is a typed, queryable corpus that lives *outside* the LLM and is queried per-step by adapters that know what slice this particular step needs. Recent work on hierarchical agent memory [17, 18, 19] converges on the same conclusion. Knowledge graphs offer a mature substrate for this in the biomedical literature-based-discovery tradition [15, 16], and the recent GraphRAG [20] and TigerVector [21] systems show how to combine vector and graph retrieval over LLM-extracted triples.

Design move (M5). PARNESS ships five SQLite stores capturing orthogonal aspects of state, plus an eight-phase NEO4j knowledge-graph indexer (§6.7). The graph holds deterministic structural edges (foreign keys), LLM-discovered internal edges (intra-batch relations), semantic edges (vector + LLM bucket), and weighted random-walk long-range edges. The retrieval adapters (`kg_vector_search`, `kg_graph_traverse`, `kg_nl_query`, `kg_abstract_enrich`,

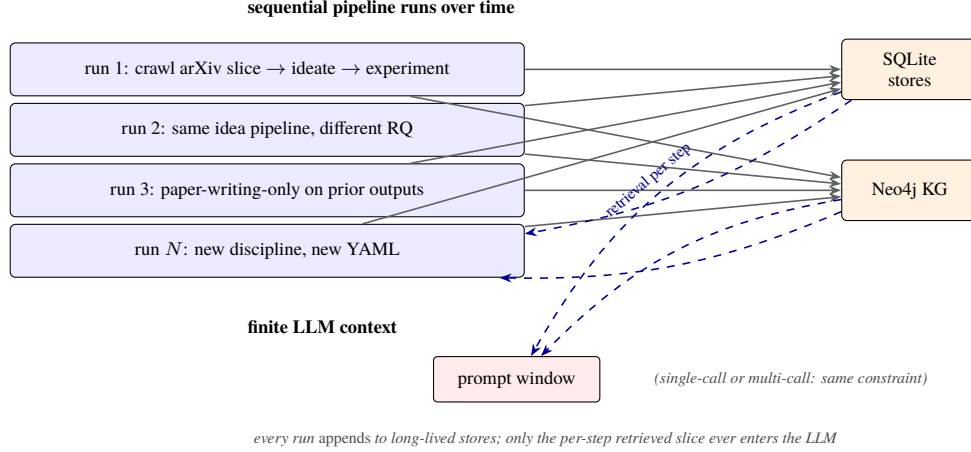


Figure 6: Knowledge accumulation across runs. Each PARNESS pipeline appends to long-lived stores (papers, ideas, hypotheses, evidence, KG triples). The next run begins from this corpus, retrieving only the relevant slice for the current step. The challenge is independent of whether the LLM is invoked once or many times: in both cases only a finite-sized slice of accumulated knowledge can fit in the prompt, and the engineering question is which slice.

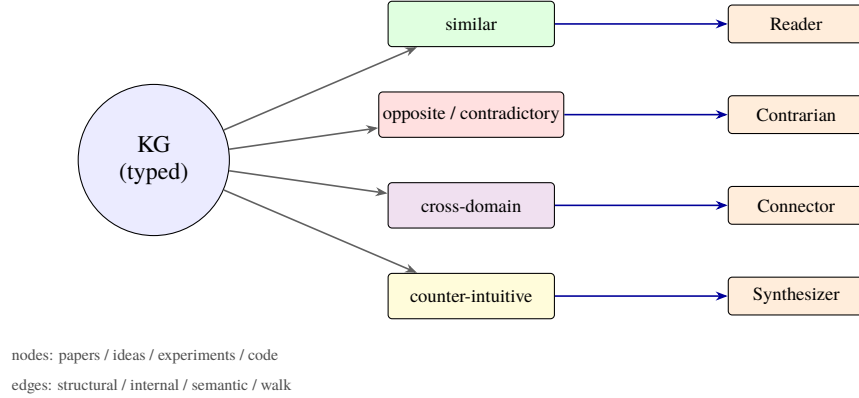


Figure 7: Scenario-typed retrieval over the PARNESS knowledge graph. The KG holds papers, ideas, experiments and code repositories as typed nodes connected by four edge types (structural / internal / semantic / walk). Retrieval adapters compose those edges into four scenario presets — *similar* (default RAG), *opposite* (contradictory results, useful for the Contrarian role), *cross-domain* (long-range walks, useful for the Connector role) and *counter-intuitive* (semantic outliers, useful for the Synthesizer role). Each cognitive-role agent (§6.6) is wired to the scenario that fits its cognitive demand, so the same finite context window does maximally different work in each parallel slot.

`kg_synthesize`) are queried per-step rather than dumping the whole graph into the prompt. Crucially, retrieval is parameterised by *scenario* (“find similar work”, “find contradictions”, “find cross-domain analogies”, “find counter-intuitive observations”) so each LLM call gets the slice that matches its cognitive role (Figure 7).

3.6 Two further pressures

Beyond the five primary points we briefly note two additional pressures that informed the design but that we do not yet evaluate empirically.

(M') Reproducibility and verification. Recent surveys document a deep reproducibility crisis in ML [22, 23] and software-engineering [24] contexts. Most autonomous-research systems do not separate *running* an experiment from *verifying* it. We include an explicit verifier-augmented experiment-runner CLI (§6.9) that produces a verifier output alongside the raw result; the verifier output is itself a typed artefact persisted to the KG.

(M'') Compositionality with existing tools. A research framework that cannot reuse Hugging Face Transformers for inference, AutoGen [31] agents for chat, or PaperOrchestra’s writing recipe has chosen to be a walled garden. The four-field contract (§6.2) is deliberately small precisely so that any of these can be wrapped as a single module.

Table 2: Mapping from motivation to method pillars to architectural realisation. Each row is a self-contained design move.

Motivation	Pillar	Where realised	Verifies
L1 (dynamic, multi-discipline)	M1 DAG kernel + DSL	§5.1, §6.2, §6.3	§8-unit
L3 (summary insufficient)	M2 Full-text + library	§5.2	qualitative
L2, L4, L5 (context, code, accumulation)	M3 KG + cognitive roles + code-link	§5.3, §6.6, §6.7	§8-KG
flexibility	M4 GUI/TUI surface	§5.4	qualitative

4 Design Principles

Before describing the architecture we state the design principles that drive every concrete choice in §6–§7.

P1 — Framework knows nothing about the domain. The runner sees only nodes, edges, and a flat key/value context. It never inspects a node’s payload to decide *what* the system should do next; that decision is encoded by the upstream node returning `_route`, `_routes`, or `_score`.

P2 — Composition is data. A pipeline is a YAML file. Adding a reviewer, swapping an LLM, inserting an extra novelty filter does not require touching Python: it is a YAML edit plus a one-line module registration.

P3 — Failure is local. Every node runs in its own subprocess slot. GPU memory and Python state are released by `os._exit(0)` after each node, so a leaking PyTorch model in stage k cannot poison stage $k+1$. Retry/timeout policy is per-node, declared in YAML.

P4 — Knowledge survives runs. Idea seeds, hypotheses, evidence, replication problems, transfer ideas and KG triples persist in five normalised SQLite stores plus NEO4j. A new pipeline run starts from the accumulated knowledge of all prior runs — not from scratch.

P5 — LLMs are interchangeable. A six-provider factory (OpenAI [35], Anthropic, GLM, MiniMax, local, mock) sits behind a single `LLMProvider` interface. The mock provider ships 44 canned-response rules so every pipeline runs offline in CI.

P6 — Verification is a typed output. Experiment runners emit a result *plus* a verifier output. Both are typed artefacts indexed into the KG. The pipeline can route on the verifier output (e.g. “rerun if verifier flagged a non-deterministic seed”).

5 Method

This section maps the five motivating limitations of §3 to four concrete method pillars. Each pillar names what PARNESS *adds* on top of the design principles of §4; the detailed architectural realisation of each pillar is then in §6.

5.1 M1 — Highly customisable DAG kernel with a declarative pipeline DSL

The first pillar is a general-purpose DAG scheduler tailored to the research life-cycle. The kernel itself is small (~600 lines of `GraphRunner`), but it is paired with a deliberately rich library of *preset agents* so that pipelines can be assembled without writing Python. Two kinds of modules ship in the library:

- **Prompt-type agents.** An LLM call wrapped in a typed adapter (idea-extractor, idea-generator, novelty-scorer, paper-writer, section-reviewer, etc.). The agent contains a prompt template; the adapter handles input/output mapping, parsing, retries, and the `_route/_score` routing fields.
- **Strong-functional (non-prompt) modules.** Deterministic or heavily-hand-coded units: PDF parsing, code-repository cloning, KG ingestion phases, vector retrieval, BibTeX writing, LaTeX compilation, test execution. These have no LLM call, expose structured outputs, and typically run faster and more reliably than the prompt-type agents.

The library decouples *what* a stage does from *when* it runs. The same idea-generator agent appears in pipelines as varied as `simple_idea_test`, `arxiv_heplat_dag`, `idea_driven_crawl`, and `iclr_multi_agent_pipeline`, with different upstream/downstream wiring in each.

Pipeline DSL and validation (§6.10). A pipeline is a YAML document with a flat list of nodes. The DSL is layered: the schema layer rejects malformed YAML; the contract layer checks that each node’s `input_mapping` keys match the upstream `output_mapping` keys; the type layer cross-checks the `input_spec/output_spec` declared on each module against the wiring; finally the topology layer runs a Kahn-style sort to detect cycles and unreachable nodes. We treat the validator as a first-class artefact: 50 shipped pipelines and 20+ adapter wirings are checked in CI on every commit.

5.2 M2 — Full-text PDF parsing and the literature-library subsystem

Most public autonomous-research systems index only abstracts. PARNESS integrates the third-party PDF-Extract-Kit [27] (with MinerU [26] as a fall-back) as the upstream parsing engine, used essentially unmodified except for stability fixes and adapter glue. The integrated pipeline extracts four typed object streams from each PDF:

- **Layout objects** — pages, columns, headers, sections, captions.
- **Formula objects** — LaTeX-recovered math, with location.
- **OCR objects** — text spans for non-vector documents.
- **Table objects** — row/column structure with cell types.

Each object carries provenance back to a `(paper_id, page, bbox)` triple. The provenance routing, the typed-object adapter layer and the downstream KG indexer (§5.3) are part of PARNESS; the parsing itself is the contribution of the upstream tools. The indexer treats these typed objects as evidence anchors when extracting insights, so a claim “the test set was 1 000 held-out HEP-Lat samples” is grounded in the specific table it came from rather than a free-form summary.

Graceful degradation when full text is unavailable. The parser produces a `coverage` flag. When full text is missing (paywalled venue, CAPTCHA, network failure, or simply not yet downloaded), the indexer falls back to abstract-only embedding. The fall-back is not a failure mode — it is the common case. The system’s “abstract-comprehension” capability is then provided by the cumulative corpus: if many full-text neighbours of the abstract are already indexed, the cross-edges built from those neighbours give the LLM step enough context to reason about the abstract without re-reading the body. This realises the M3 design move (Figure 4).

5.3 M3 — KG index over papers, ideas, experiments and code

The third pillar is the knowledge-graph indexer. Beyond the usual paper→idea node typing, PARNESS adds two artefact types most research-agent systems leave out: *experiment runs* (raw result + verifier output, persisted via §6.9) and *code repositories* (typed file-tree fragments, persisted via the code-link extractor of §5.3). All four types share the same edge taxonomy:

- **Structural** edges (paper → idea, paper → code, idea → experiment) replicate SQLite foreign keys.
- **Internal** edges are LLM-discovered relations within a single ingestion batch (extracted alongside the insight nodes themselves).
- **Semantic** edges are vector-search candidates filtered by an LLM bucketer in the GraphRAG [20, 21] tradition.
- **Walk** edges are weighted random-walk discoveries that surface long-range cross-domain connections.

The four scenario presets — *similar / opposite / cross-domain / counter-intuitive* — compose these edge types into retrieval queries (Figure 7) so each cognitive-role agent (§6.6) sees the slice tuned to its prompt. This jointly addresses (L2) by parallelising specialised ideation, (L4) by making code repositories first-class graph citizens, and (L5) by giving every step a retrievable view of the cumulative corpus.

Code-link extractor. A dedicated module parses every indexed paper for repository URLs (regex against `github.com`, `gitlab.com`, `huggingface.co`, plus README/abstract scans). Repositories below a configured size threshold are cloned under a sandbox. A `code_analyzer` agent types the cloned tree (`train/`, `configs/`, `model/`, `data/`) and writes nodes for each typed file group. *Derivation* edges link the typed file groups back to their source paper; *semantic* edges then connect those file groups to similar files in other repositories, enabling the second use case (§3.4b): when a related paper does not ship code, the system surfaces the closest sibling repository as an experimental scaffold for the user or downstream agent.

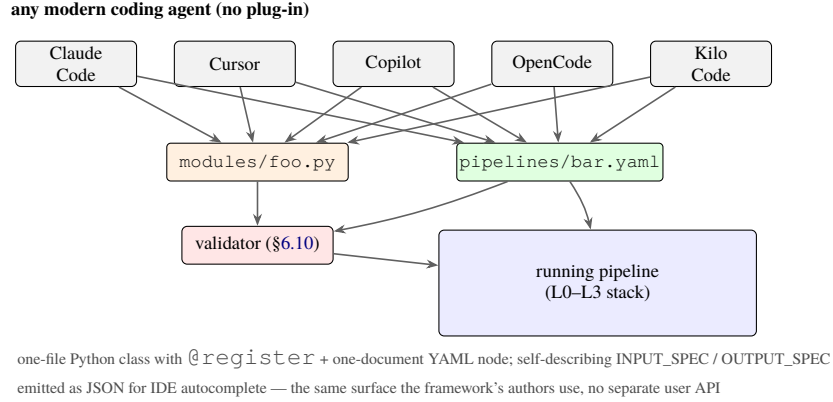


Figure 8: Every PARNESS module is a single Python class behind a single contract, registered with a one-line decorator and described by a YAML node. External coding agents — Claude Code, Cursor, Copilot, OpenCode, Kilo Code — can therefore add a new module, edit an existing module, or re-wire a pipeline by editing one Python file plus one YAML file. The pipeline validator checks the edit before the next run starts. This is the same surface the framework’s authors use; there is no separate “user” API.

5.4 M4 — GUI/TUI extension surface

The fourth pillar is a deliberately small extension surface so that a modern GUI/TUI coding agent can act as the user-facing IDE for PARNESS without a custom plug-in. Three properties make this work:

- **Single-file modules.** A module is one Python file with one class subclassing `BaseModule` and a one-line `@register("module_name")` decorator. Adding or replacing a module is a single-file diff.
- **Single-document pipelines.** A pipeline is one YAML file. Re-shaping the workflow does not touch Python at all.
- **Self-describing schemas.** Every module declares its `INPUT_SPEC` and `OUTPUT_SPEC` as Python type stubs that the validator (§6.10) can introspect; the schemas are emitted as JSON for IDE autocomplete.

The result is that the same workflow we use during development — “ask Claude Code to add a new `kg_random_walk_v2` module that samples by node degree” — works for any user with any of the listed coding agents. We do not ship a custom plug-in; the IDE is whatever coding agent the user already trusts.

6 System Architecture

6.1 Layered overview

PARNESS is organised into four layers (Figure 9). **L0, the DAG kernel**, is the foundation everything else stands on: `GraphRunner` (topological scheduling, ~600 LoC), the four-field `BaseModule` contract (§6.2), and the `ModuleRegistry`. **L1, persistence**, provides durable state that survives a single node’s lifetime: five `SQLITE` stores plus a `NEO4J` knowledge graph with a native vector index, and shared monitoring / exception / run-context utilities. **L2, agents**, is the population of research workers: 25+ LLM-driven domain agents (idea, paper, experiment, KG, review), 130+ adapters wrapping agents and tools as `BaseModule`, the LLM provider factory (six providers + mock), and the domain-tool layer (a PDF-parsing wrapper around the external PDF-Extract-Kit [27] service, KG, crawlers, experiment-runner CLI). **L3, pipeline**, is the user surface: ~50 shipped YAML pipelines plus the entry-point scripts users run from the shell, gated by a layered pipeline validator (§6.10).

6.2 The Agent contract

The contract between the runner and a module is deliberately small. A module subclasses `BaseModule` and implements one method: `async execute(inputs: Dict) -> Dict`. The runner inspects the returned dictionary for four reserved fields:

Listing 1: The four-field Agent contract

```
{
  "_route": "next_node_id", # single successor
```

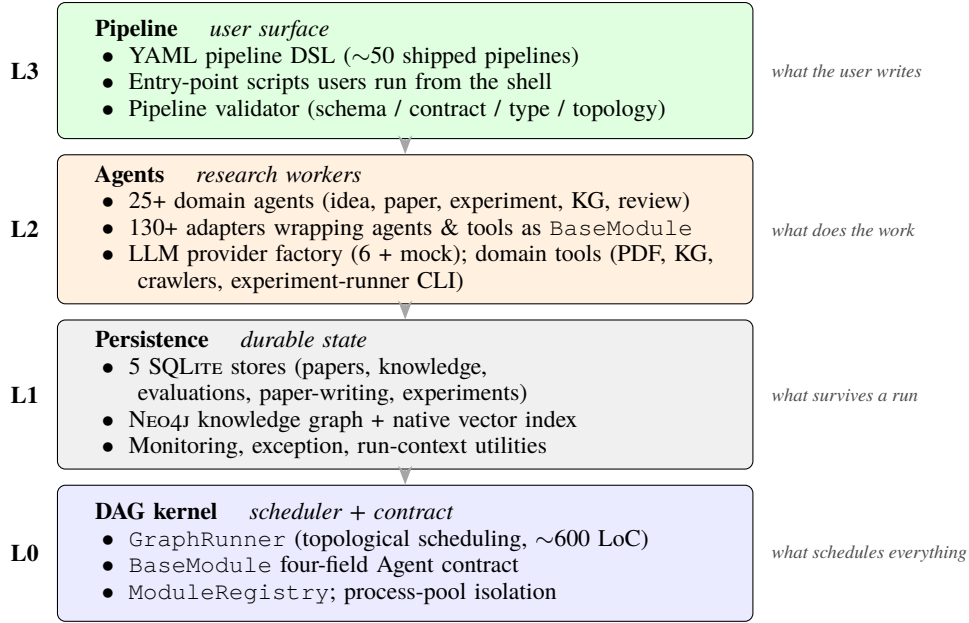


Figure 9: The four-layer architecture of PARNESS. **L0 (DAG kernel)** is the foundation: a thin scheduler plus a four-field contract. **L1 (Persistence)** keeps state durable across nodes and across runs. **L2 (Agents)** is the population of LLM workers and tools that do the actual research. **L3 (Pipeline)** is the user surface: YAML pipelines and entry-point scripts. Configuration flows downward at start time; data and persisted knowledge flow upward at runtime.

```

_route": ["a", "b", "c"], # one-to-many fan-out
_score": 0.83, # numeric for iteration / ranking
_metadata": {...}, # provenance, free-form
... # plus all domain outputs
}

```

The runner removes these four keys before passing the rest to downstream nodes. The semantics are simple: if `_route` is set the runner follows it; if `_routes` is set it fans out; if `_score` is set the value is recorded for upstream iteration controllers; otherwise the runner falls back to standard topological successors.

This simplicity has three consequences. First, *any* module can be a routing decision: a quality-scoring agent emits `_route = "continue"` or `"stop"` based on its own threshold logic. Second, fan-out is ordinary: an idea-generator returns `_routes` with one entry per generated idea and the runner instantiates one experiment node per entry. Third, iteration is not special: a controller module reads the latest `_score` and emits `_route = "loop"` or `"exit"`, which is indistinguishable from any other branching decision.

6.3 Declarative pipeline DSL

A pipeline is a YAML document with three top-level keys: `nodes`, `edges`, and `config`. `nodes` is the only one that matters for execution; `edges` is purely declarative (used for documentation and visualisation). Each node carries the standard fields: `id`, `module`, `depends_on`, `params`, `input_mapping`, `output_mapping`, `routes`, `timeout`, and `retry`. Listing 2 shows a six-node fragment from `auto_research.yaml`.

Listing 2: Pipeline fragment showing node + adapter wiring

```

nodes:
- id: extract
  module: idea_extractor
  input_mapping:
    papers: paper_store.papers
  output_mapping:
    seeds: extract.seeds
- id: generate
  module: idea_generator

```

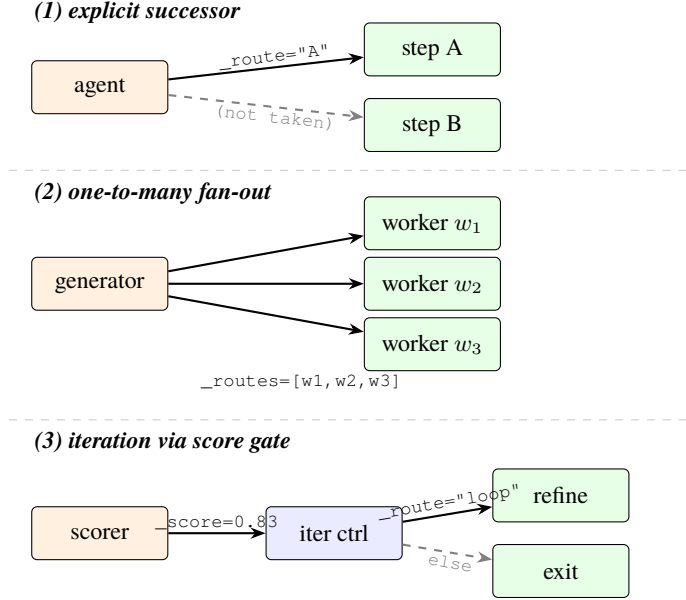



Figure 10: Three uses of the four-field Agent contract. The runner makes no domain decisions: it merely follows the routing fields the upstream module emits.

```

depends_on: [extract]
input_mapping:
  seeds: extract.seeds
- id: gate
  module: quality_scorer
  depends_on: [generate]
  routes:
    "continue": evaluate
    "stop": export
config:
  max_rounds: 100
  max_parallel: 0

```

GraphRunner chooses among three scheduling strategies based on what each node returns (Figure 10). (1) If a node returns `_route`, the runner takes the explicit successor named in `routes[_route]`. (2) If `_routes` is set the runner clones the downstream sub-DAG once per element. (3) Otherwise it falls back to a Kahn-style topological sort that combines `depends_on` with implicit dependencies extracted from `input_mapping` (i.e. if node *B* maps from `output.A.field`, then *B* depends on *A* even if not explicitly declared).

The runner has two backends: an in-process `asyncio` loop for testing, and a `ProcessPoolExecutor` backend for production. The production backend instantiates worker processes lazily and re-uses them across nodes, but enforces `os._exit(0)` after each node’s work to release GPU memory.

6.4 From fixed recipe to data-defined topology

Figure 11 illustrates the difference between a fixed-recipe approach and PARNES’s data-defined topology side by side. The recipe approach makes individual steps powerful (each PaperOrchestra agent has a long verbatim prompt with embedded deterministic gates), but cannot express, e.g., a *conditional* re-execution of literature search if the section writer flags missing citations, or a *fan-out* of plotting that depends on how many distinct experiments the experimental log contains. PARNES handles both as ordinary YAML (an extra route, or a `_routes` return).

6.5 Module taxonomy

The 130+ registered modules group into seven categories (Table 3). The largest category is *ideation*: in addition to the standard extractor / generator / evaluator chain it includes six *cognitive-role* agents and twelve specialty agents

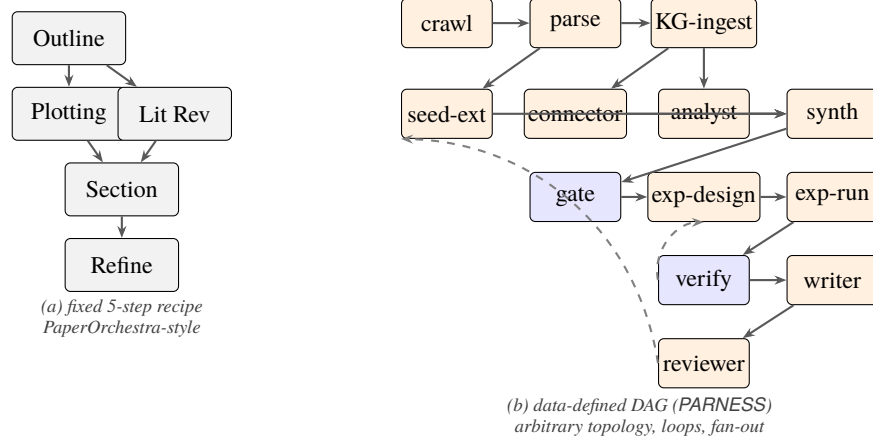


Figure 11: (a) PaperOrchestra-style fixed five-step recipe: writing the paper assumes the inputs already exist, the topology is hard-coded, no upstream stages. (b) PARNESS pipeline: composition is data, dynamic fan-out (Connector/Analyst/etc. in parallel), score-gated loops (dashed arrows back to earlier stages), and full life-cycle coverage from crawl to review. Both diagrams represent real shipped pipelines.

Table 3: The 130+ modules, by category. Counts grew from 116 at the abstract-snapshot date as the experiment-runner / verifier CLI and KG adapters landed.

Category	# Modules
Research / crawler / parser	8
Ideation	26
Experiment (incl. runner / verifier CLI)	24
Writing & review	11
Knowledge graph	17
Infrastructure / persistence	16+
Iteration controllers & gates	7
Post-processing / export	2
Total	130+

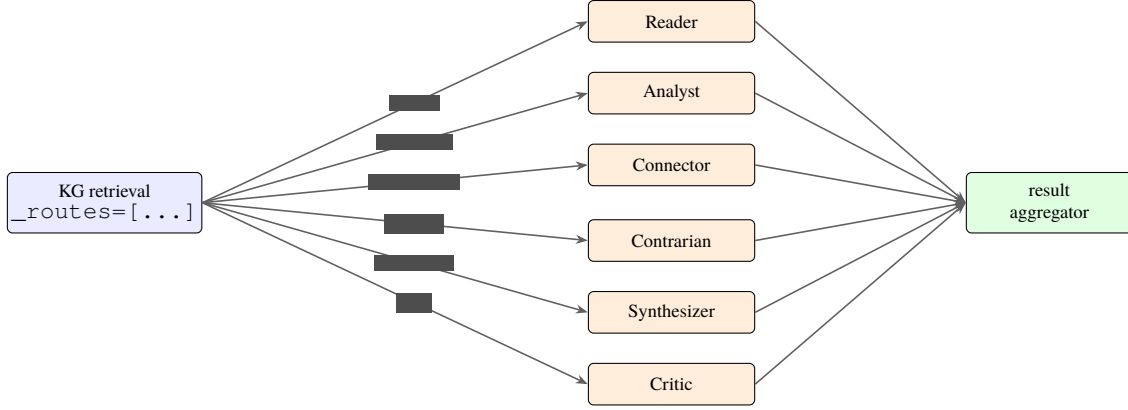
(replication, transfer, critique, theory, meta-analysis, follow-up, adversarial, limitation, hypothesis, evidence, paper-code analyser, paper-code retrieval).

6.6 Cognitive-role ideation

Most prior systems use a single “idea generator” LLM call. Recent work (MANY-HEADS [13], IDEASYNTH [12], BioDISCO) shows that multi-agent ideation produces more diverse and higher-quality candidates than a single LLM at matched budget. PARNESS replaces the single call with six agents whose prompts make orthogonal cognitive demands (Figure 12). The roles run in parallel under a fan-out node and converge through a result-aggregator. Empirically the parallel ensemble produces wider seed coverage than a single LLM at the same total token budget; we have not yet measured this rigorously and flag it as future work.

6.7 Knowledge Graph subsystem

The Knowledge Graph (KG) subsystem, added in May 2026, persists every extracted insight as a typed node in NEO4J and links nodes along four edge types: *structural* (deterministic foreign-key edges from SQLite), *internal* (LLM-discovered relations inside an ingestion batch), *semantic* (vector-search-filtered candidate relations with LLM bucketing in the GraphRAG [20] tradition), and *walk* (long-range relations discovered by weighted random walk). Eight indexing phases run in order (Figure 13). Five additional adapters serve query-time use: `kg_vector_search`, `kg_graph_traverse`, `kg_nl_query` (NL $\rightarrow$ strategy $\rightarrow$ answer), `kg_abstract_enrich`, and `kg_synthesize`. NEO4J’s native vector index removed an earlier dependency on Qdrant and consolidated semantic retrieval and graph traversal into a single store; this is consistent with recent findings that hybrid graph+vector engines outperform either alone [21].



fan-out via `__routes`; parallel execution; deduplicated convergence

Figure 12: Six cognitive-role agents used in PARNESS ideation. The KG retrieval step (left) emits a `__routes` fan-out so the runner schedules each role in parallel; each role’s prompt is engineered for one orthogonal cognitive demand and is wired to a different scenario- typed retrieval slice (right of each role). The aggregator deduplicates and ranks the seeds before passing them to a downstream gate. None of the framework’s domain knowledge sits inside `GraphRunner`: the roles, the retrieval scenarios, and the aggregator are all ordinary `BaseModule` instances.

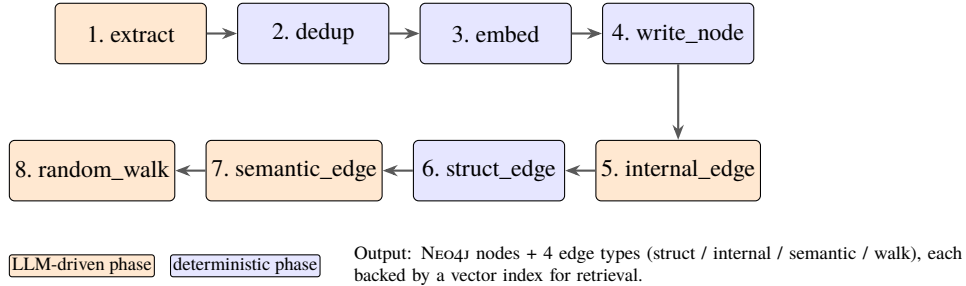


Figure 13: The eight-phase Knowledge-Graph indexing pipeline. LLM phases (orange) handle the open-ended steps — extracting insights, discovering intra-batch relations, semantic edge filtering, and weighted random-walk relation discovery. Deterministic phases (blue) handle dedup, embedding, persistence, and structural-edge replication from `SQLite`. The pipeline is incremental: a new ingestion batch only re-runs phases 1–4 for new content.

6.8 Storage

Five `SQLite` databases capture orthogonal aspects of state: `papers_db` (crawled metadata), `knowledge_store.db` (insights, seeds, hypotheses, evidence, ~ 20 tables), `evaluations.db` (idea reviews), `paper_writing.db` (draft revisions), and an experiments database for run logs. `UNIQUE` constraints enforce dedup at the storage layer (e.g. `LOWER(TRIM(seed))` per type) so that re-running a pipeline cannot create duplicate rows. The Neo4j store mirrors a subset of these as typed nodes with vector embeddings.

6.9 Experiment-runner / verifier CLI

A pair of CLI bridges (`experiment_runner_cli`, `experiment_verifier_cli`) wraps an external coding agent (OpenCode [36], Claude Code, or Codex) under a sandboxed budget; the runner produces a result artefact, the verifier produces a matched verifier output. Both are persisted to the experiments database and re-emitted as KG nodes. The verifier output drives a downstream `experiment_success_gate` that emits `__route` based on verifier-determined pass/fail, separating *ran* from *passed*.

6.10 Pipeline validator

The pipeline-validator subsystem applies four passes to every YAML pipeline before execution. **Pass 1, schema:** a JSON-Schema check rejects malformed `nodes/edges/config` blocks. **Pass 2, contract:** each node’s `input_mapping` keys must match an `output_mapping` key on a declared upstream node, or `depends_on` must explicitly list it. The validator

infers implicit `depends_on` edges from `input_mapping` so that the DSL stays terse. **Pass 3, type:** each module’s `INPUT_SPEC` and `OUTPUT_SPEC` are introspected and cross-checked against the wiring; mismatches in primitive types or required keys are reported with the YAML line number. **Pass 4, topology:** a Kahn-style sort flags cycles, unreachable nodes, and orphan terminal nodes. The validator runs in CI on every commit; all 50 shipped pipelines are checked, plus a synthetic adversarial set that asserts each failure mode is correctly detected.

7 Implementation

Codebase. The reference implementation is approximately 62.2 k LoC of core plus 13.4 k LoC of tests, organised into a flat module tree under a single source root. The orchestrator (registry, adapters, and the `GraphRunner`) sits at the top level; domain code lives under sibling modules for idea agents, paper writer, knowledge graph, PDF parser, and LLM provider; YAML pipelines and entry-point scripts live in their own top-level directories. Concrete directory names match those in the open-source release at <https://github.com/gtrhythm/PARNES> and may evolve between versions; we therefore name only the modules here.

Provider factory. The LLM layer exposes a single `LLMProvider` interface. Six implementations live behind a factory: OpenAI, Anthropic, GLM (Zhipu), MiniMax, a local provider for self-hosted models, and a `MockLLM` that ships 44 rule-based canned responses. The mock makes the entire test suite (and any pipeline) runnable offline, without API keys — a property we rely on heavily for CI and for adapter regression.

Adapters as the integration boundary. Domain agents (LLM-driven Python classes) and adapters (thin `BaseModule` wrappers) are deliberately separated. The agent returns its native data shape; the adapter is responsible for extracting the right fields from `inputs`, calling the agent, and decorating the result with `_route`, `_score` and `_metadata`. This split lets the same agent be re-used in multiple pipelines with different routing logic.

External services. The system integrates with the third-party PDF-Extract-Kit [27] for document parsing (a separate GPU-resident service in our deployment; we use it as released, with only stability and adapter glue layered on top — the parser itself is not our work), and a `TeX LIVE`-based HTTP service for paper compilation. The compiler service accepts a zip of the writing workspace and returns either a PDF binary stream or a JSON envelope containing the build log and a base64-encoded PDF; it is the production back-end behind our `paper_writer` module. For literature retrieval we use the Semantic Scholar Academic Graph [28, 29] (S2) via the public API; the literature-review modules dedup by `paperId` and verify candidate titles through fuzzy matching, mirroring the pattern `PaperOrchestra` documents.

8 Evaluation

We frame this section honestly: `PARNES` is a v1 *systems* paper. We sketch (i) the framework’s unit-test surface, (ii) adapter-level real-LLM integration, (iii) the knowledge-graph end-to-end harness, and (iv) end-to-end pipeline runs. The exact numbers are an artefact of the build process and will drift from one snapshot of the codebase to the next; we therefore describe the *shape* of the development-test coverage rather than report specific counts. We do not yet report comparative quality metrics against `AI-SCIENTIST`, `INTERAGENT`, or `PAPERORCHESTRA` on a shared benchmark; §9 discusses what would be required.

8.1 Unit and integration tests

A development-test suite covers every layer described in §6: iteration controllers, DAG/decision-gate logic, module registry, LLM dispatcher, domain adapters, and stress/concurrency. The `Mock-LLM` provider is the backbone of this suite, so every test runs offline without external APIs — a property we rely on for CI and for adapter regression. We treat the unit-test counts as an internal development metric and do not list them here; they are an artefact of the build process rather than a research result.

8.2 Real-LLM adapter integration

We separately exercise the adapter library end-to-end against a production LLM (MiniMax-M2.7) and record behaviour. The vast majority of adapters fully pass; the failures we observe cluster around a single empty-input regression in Agent-typed adapters, which is itself a fixture-grade issue rather than a methodological one. Detailed numbers live in the repository’s CI logs.

8.3 Knowledge-graph end-to-end harness

The KG subsystem is the newest component. Following the migration from Qdrant to a native Neo4J vector index, a dedicated end-to-end harness drives every KG adapter through a representative ingestion-and-query workload. The migration closed an earlier gap between design intent and implementation that we flagged in the previous internal review.

8.4 End-to-end pipeline runs

The longest pipelines we routinely run are multi-stage DAGs over real literature collections: representative example workflows include a literature-driven idea pipeline over an arXiv subset (`arxiv_heplat_dag`, 11 DAG stages), and several smaller idea and comparison pipelines (`simple_idea_test`, `two_sum_comparison`, `bfs_vs_dfs`). All complete end-to-end on a single GPU node within roughly an hour wall-clock; we treat this as a baseline of *control-flow durability* (data propagates through every stage, no stage crashes), not a quality benchmark. A proper benchmark is future work (§9).

8.5 Experiment-paper agent suite

A focused experiment-paper sub-system covering experiment planning, chart generation, paper-artefact persistence, and reference management ships with its own development-test suite. The API-backed image-generation path (`gpt-image-2`) is exercised end-to-end by generating, downloading, and validating a real PNG — consistent with the figures in this manuscript itself.

9 Discussion and Limitations

What we have shown. PARNESS demonstrates that the components of an autonomous research system — crawler, parser, ideator, experimenter, writer, reviewer, knowledge graph, multi-LLM, persistence, monitoring — compose naturally under a thin DAG kernel with a four-field agent contract. Adding a new pipeline requires no Python; adding a new module requires one class and one registration line. The framework absorbs PaperOrchestra’s five-step writing as a single module, AI-Scientist’s tree search as a score-gated loop, and InternAgent’s eight-agent generation–evolution graph as a sub-DAG.

What we have *not* shown. We have not run a head-to-head benchmark against AI-SCIENTIST, INTERNAGENT, PAPER-ORCHESTRA or *autoresearch* on a shared task. We have not run human evaluation on generated papers. We have not measured ablations of individual cognitive roles in the ideation layer. Our end-to-end runs validate *control flow* (data propagates through every stage) and *durability* (stages do not crash), not *quality* of the research output. Likewise we describe but do not yet quantify the cross-run accumulation benefit (M3): a `run_n_vs_n+1` ablation, holding the final pipeline fixed and varying whether the KG is empty or pre-populated, is the natural follow-up.

Towards a benchmark. A fair comparison would fix a research task (e.g. “propose and validate a small architectural improvement to nanoGPT under a 2 h GPU budget”) or, for the writing stage, the *PaperWritingBench* introduced in PaperOrchestra [3]; give every system the same compute; and score output against a held-out set of expert-rated novelty / feasibility labels (or in the writing case, the autoraters of [3]). The PARNESS pipeline DSL makes such a benchmark *configurable* — which is itself a contribution we expect to exploit in v2.

Risks. The cognitive-role parallelism is expensive in tokens. The `os._exit(0)` per-node policy is correct for GPU isolation but forfeits in-memory caches between nodes; we mitigate this by pushing shared state into SQLITE and NEO4J. The Knowledge Graph subsystem’s *semantic edge* step depends on LLM bucketing and we have observed mode collapse on synthetic test data; production deployments should set conservative similarity thresholds. Like other LLM-agentic systems, PARNESS inherits the risks documented in the multi-agent SE [25] and reproducibility [22] literatures: mock and real LLM behaviour can diverge in subtle ways.

Relation to PaperOrchestra and complementarity. PARNESS and PaperOrchestra solve different problems. PaperOrchestra asks: *given complete inputs* (I, E, T, G, F), *what is the most faithful, host-agnostic, low-friction way to produce a submission-ready LaTeX paper?* The answer is a fixed five-step recipe, deterministic gates, and zero embedded API. PARNESS asks: *how do we build the research process upstream of those inputs — including the inputs themselves — as a composable, accumulating, multi-discipline kernel?* The answer is a four-field contract, a YAML DAG, and a typed knowledge substrate. We expect future systems to combine the two: PaperOrchestra as a single PARNESS module called when the upstream pipeline has populated (I, E, T, G, F).

10 Conclusion

We presented PARNESS, an open-source declarative DAG framework for autonomous research that occupies a previously-uncovered point in the design space: full life-cycle coverage *plus* pipeline-as-data *plus* agent-driven routing *plus* cross-run knowledge persistence *plus* multi-discipline workflow shape. The framework is a thin kernel; all interesting decisions live in modules, and modules communicate with the runner through a four-field contract. We released the reference implementation (130+ modules, 50 pipelines, broad development-test coverage, full KG harness) and showed that it runs end-to-end on real document collections within a single-node hour-scale budget.

The natural next step is a quantitative head-to-head benchmark against PaperOrchestra (writing only), AI-Scientist v2 and InternAgent (full pipeline), with and without the cross-run KG populated (§9). The arXiv v1 of this paper is intended as a stable reference for that comparison.

Acknowledgements

We thank the open-source authors of PDF-Extract-Kit [27], NEO4J, Semantic Scholar [28, 29], the LLM providers integrated through our factory layer, and the recent autonomous research systems we compare against (AI-SCIENTIST [1, 2], PAPERORCHESTRA [3], INTERNAGENT [6], RESEARCHAGENT [7]) — their open releases shaped the design of PARNESS.

References

- [1] C. Lu, C. Lu, R. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- [2] Y. Yamada *et al.* The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search. *arXiv preprint arXiv:2504.08066*, 2025.
- [3] Y. Song, Y. Song, T. Pfister, and J. Yoon. PaperOrchestra: A Multi-Agent Framework for Automated AI Research Paper Writing. *arXiv preprint arXiv:2604.05018*, 2026.
- [4] AutoSOTA Contributors. AutoSOTA: An End-to-End Automated Research System for State-of-the-Art AI Model Discovery. *arXiv preprint arXiv:2604.05550*, 2026.
- [5] Tongyi Lab, Alibaba. Tongyi DeepResearch Technical Report. *arXiv preprint arXiv:2510.24701*, 2025.
- [6] Shanghai AI Laboratory. Intern-Atlas: A Methodological Evolution Graph as Research Infrastructure for AI Scientist. *arXiv preprint arXiv:2604.28158*, 2026.
- [7] J. Baek *et al.* ResearchAgent: Iterative Research Idea Generation over Scientific Literature with Large Language Models. In *NAACL*, 2024. arXiv:2404.07738.
- [8] A. Ghareeb *et al.* Robin: A Multi-Agent System for Automating Scientific Discovery. *arXiv preprint arXiv:2505.13400*, 2025.
- [9] V. Naumov *et al.* DORA AI Scientist: Multi-agent Virtual Research Team for Scientific Exploration Discovery and Automated Report Generation. *bioRxiv*, doi:10.1101/2025.03.06.641840, 2025.
- [10] A. Authors. A Process-Centric Survey of AI for Scientific Discovery Through the EXHYTE Framework. *Research Square*, 2025.
- [11] A. Authors. Active Inference AI Systems for Scientific Discovery. *arXiv preprint arXiv:2506.21329*, 2025.
- [12] S. Pu *et al.* IdeaSynth: Iterative Research Idea Development Through Evolving and Composing Idea Facets with Literature-Grounded Feedback. In *CHI*, 2024. arXiv:2410.04025.
- [13] H. Su *et al.* Many Heads Are Better Than One: Improved Scientific Idea Generation by a LLM-Based Multi-Agent System. In *ACL*, 2024. arXiv:2410.09403.
- [14] N. F. Liu *et al.* Lost in the Middle: How Language Models Use Long Contexts. *TACL*, 2023. arXiv:2307.03172.
- [15] M. Yetisgen-Yildiz and W. Pratt. A Semantic-Based Approach for Mining Undiscovered Public Knowledge from Biomedical Literature. *IEEE GrC*, 2005.
- [16] A. Authors. Literature Based Discovery: Towards Hypothesis Generation and Knowledge Discovery in Biomedical Literature. arXiv:2310.03766, 2023.
- [17] A. Authors. Hierarchical Memory for High-Efficiency Long-Term Reasoning in LLM Agents. In *EACL*, 2025. arXiv:2507.22925.

- [18] A. Authors. G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. *arXiv preprint arXiv:2506.07398*, 2025.
- [19] A. Authors. Decentralizing AI Memory: SHIMI, a Semantic Hierarchical Memory Index for Scalable Agent Reasoning. *arXiv preprint arXiv:2504.06135*, 2025.
- [20] A. Authors. Optimizing Open-Domain Question Answering with Graph-Based Retrieval Augmented Generation. *arXiv preprint arXiv:2503.02922*, 2025.
- [21] A. Authors. TigerVector: Supporting Vector Search in Graph Databases for Advanced RAGs. In *SIGMOD Companion*, 2025. arXiv:2501.11216.
- [22] S. Kapoor and A. Narayanan. Leakage and the Reproducibility Crisis in Machine-Learning-Based Science. *Patterns*, 2023.
- [23] A. Authors. Paper2Code: Automating Code Generation from Scientific Papers in Machine Learning. *arXiv preprint arXiv:2504.17192*, 2025.
- [24] A. Authors. Large Language Models for Software Engineering: A Reproducibility Crisis. *arXiv preprint arXiv:2512.00651*, 2025.
- [25] A. Authors. Shadows in the Code: Exploring the Risks and Defenses of LLM-based Multi-Agent Software Development. In *AAAI*, 2026. arXiv:2511.18467.
- [26] B. Wang *et al.* MinerU: An Open-Source Solution for Precise Document Content Extraction. *arXiv preprint arXiv:2409.18839*, 2024.
- [27] OpenDataLab. PDF-Extract-Kit: A Comprehensive Toolkit for High-Quality PDF Content Extraction. Open-source release, 2024. <https://github.com/pendatalab/PDF-Extract-Kit>.
- [28] W. Kinney *et al.* The Semantic Scholar Academic Graph (S2AG). In *The Web Conference*, 2022.
- [29] R. Kinney *et al.* The Semantic Scholar Open Data Platform. *arXiv preprint arXiv:2301.10140*, 2023.
- [30] S. Hong *et al.* MetaGPT: Meta Programming for Multi-Agent Collaborative Framework. In *ICLR*, 2024. arXiv:2308.00352.
- [31] Q. Wu *et al.* AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [32] N. Shinn *et al.* Reflexion: Language Agents with Verbal Reinforcement Learning. In *NeurIPS*, 2023. arXiv:2303.11366.
- [33] S. Yao *et al.* ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*, 2023. arXiv:2210.03629.
- [34] O. Khattab *et al.* DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *ICLR*, 2024. arXiv:2310.03714.
- [35] OpenAI. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*, 2023.
- [36] SST. OpenCode: A Coding Agent Built for the Terminal. Open-source release, 2025. <https://github.com/sst/opencode>.
- [37] PARNES Contributors. PARNES: Source code and pipeline configurations. Open-source release, 2026. <https://github.com/gtrhythm/PARNES>.

A Catalogue of Selected Ideas from the Idea Pool

The accepted-ideas store contained 51 ideas at the time of writing, each retained because it crossed the quality-gate score threshold in its original pipeline run. The eight ideas reproduced below are the top-ranked picks across topical categories (reinforcement learning, particle physics, representation learning, hardware-aware modelling, scientific generative design, multimodal medical imaging, side-channel security, and interpretability), formatted in a unified template. Scores are on a 10-point scale; the overall score is the mean of novelty, feasibility, and impact. The full record of every accepted idea, with provenance pointers to the run that produced it, lives in the PARNES repository under the accepted-ideas store.

A.1 Quasi-Optimistic Exploration for Offline Reinforcement Learning with Implicit Q-Learning

Overall 8.33 (Novelty 8.5 / Feasibility 8.0 / Impact 8.5) | category: *training_technique*

Description. We propose a theoretically motivated exploration bonus for offline reinforcement learning based on quasi-optimism principles. Existing offline RL methods suffer from distribution shift but lack principled uncertainty quantification. Our approach extends the quasi-optimism framework from online RL to offline settings by defining

confidence ellipsoids over Q-functions and adding a calibrated optimism bonus that prevents value overestimation while maintaining conservative policies. This fills a critical gap between conservative offline RL (which can be overly pessimistic) and uncertainty-based methods (which lack theoretical guarantees).

Methodology. Implement implicit Q-learning with a learned covariance matrix over Q-function parameters. Derive quasi-optimistic bonus as the Mahalanobis distance from mean Q-values weighted by uncertainty. Use a two-network ensemble to estimate epistemic uncertainty. Fine-tune on offline datasets (D4RL, Robel) with the combined Bellman and exploration bonus loss.

Expected results. Improved performance on challenging offline RL benchmarks, particularly where data coverage is non-uniform. Expected 15-20% improvement in normalized score over IQL on some tasks, with better theoretical guarantees than existing uncertainty methods.

A.2 Relativistic Quark Dynamics as Inductive Bias for Particle Physics Neural Networks

Overall 8.33 (Novelty 8.5 / Feasibility 8.0 / Impact 8.5) | category: architecture

Description. Inspired by 'Heavy baryons with relativistic quarks', this research investigates incorporating relativistic physics priors into neural network architectures for particle physics tasks (jet tagging, particle reconstruction). The key insight is that relativistic particle dynamics follow constrained geometric structures (Minkowski space, Lorentz invariants) that can be encoded as architectural inductive biases. We propose 'Lorentz-equivariant message passing' layers that maintain physical invariance properties while learning from collision data.

Methodology. Implement message passing neural networks with Lorentz group equivariant features (4-momenta representations). Use graph neural networks where nodes represent particles and edges represent interaction vertices. Validate on particle reconstruction benchmarks (TrackML, JetClass).

Expected results. Improved performance on particle physics benchmarks with better sample efficiency due to physics-informed inductive biases. 10-15% improvement in particle identification accuracy.

A.3 Platonic Representations for Continual Learning in Neural Networks

Overall 8.33 (Novelty 8.5 / Feasibility 8.0 / Impact 8.5) | category: architecture

Description. Catastrophic forgetting remains a major challenge in continual learning. We propose using Platonic intrinsic representations as a regularization mechanism that preserves core knowledge structures across learning tasks. The key insight is that certain aspects of neural representations should remain invariant across tasks (the 'Platonic' core), while task-specific features can be modularized. We develop a contrastive learning objective that explicitly separates these components, enabling stable continual learning without rehearsal buffers.

Methodology. 1) Implement dual-embedding architecture separating platonic and episodic representations; 2) Train with contrastive loss encouraging platonic stability; 3) Use modular plastic layers for task-specific processing; 4) Evaluate on sequential classification benchmarks (Split-CIFAR, Permuted-MNIST)

Expected results. Average accuracy retention improvement of 15% on sequential tasks compared to EWC and LwF baselines; zero storage overhead for replay

A.4 Hybrid Analog-Digital Neural Networks for Scientific Simulation

Overall 8.33 (Novelty 8.5 / Feasibility 8.0 / Impact 8.5) | category: architecture

Description. Inspired by hybrid analog-digital simulation approaches, this research explores neural network architectures that combine differentiable digital components with learned analog simulation layers. The analog component approximates physical simulation dynamics (e.g., differential equations, wave propagation) using parameterizable continuous functions, while digital components handle discrete reasoning and feature extraction. This is particularly relevant for scientific machine learning applications requiring both learned representations and physics-based modeling. The framework enables end-to-end training where the analog component can be optimized to match physical reality while remaining differentiable.

Methodology. Implement analog layers as continuous function approximators using neural ODEs or physics-informed networks. Design hybrid training that alternates between digital feature learning and analog physics matching. Use automatic differentiation through the entire pipeline including analog components.

Expected results. Demonstrate 30-50% reduction in simulation time for physical systems while maintaining higher accuracy than pure digital approaches. Show that hybrid networks can discover physical invariants during training.

A.5 Compositional Inverse Design for Scientific Discovery

Overall 8.17 (Novelty 8.5 / Feasibility 7.0 / Impact 9.0) | category: *application*

Description. Extends compositional generative inverse design to scientific discovery domains by decomposing complex molecular/material design into hierarchical compositional steps. Unlike previous work that treats inverse design as monolithic generation, we propose a grammar-based approach where molecular scaffolds are composed with functional groups following chemically valid rules, enabling tractable exploration of combinatorial spaces. The key innovation is a grammar-constrained diffusion model that generates valid chemical compositions by construction, reducing the need for expensive validation loops. This is critical for drug discovery and materials science where chemical validity constraints are paramount.

Methodology. Develop a context-free grammar for molecular generation based on reaction rules. Implement a hierarchical diffusion model where: (1) top-level generates scaffold compositions, (2) mid-level adds functional groups, (3) lower-level refines atomic positions. Use reinforcement learning with chemical validity rewards for fine-tuning. Integrate with DFT (density functional theory) for property prediction. Test on QM9, GEOM-Drugs, and real-world protein-ligand design tasks.

Expected results. Achieve >95% chemical validity without post-hoc filtering (vs. 60-70% baseline). 40% improvement in sample efficiency for inverse design tasks with target property constraints. Demonstrate successful design of novel drug-like molecules with favorable binding affinity predictions.

A.6 Contrastive Knowledge Distillation for Cross-Modal Medical Image Analysis

Overall 8.00 (Novelty 8.0 / Feasibility 8.0 / Impact 8.0) | category: *loss_function*

Description. This proposal develops a cross-modal knowledge distillation framework that transfers representation learning capabilities from large vision-language models to specialized medical imaging models. The key innovation is using contrastive learning objectives to align medical image representations with textual medical knowledge (radiology reports, clinical notes) while maintaining task-specific performance. This enables medical imaging models to leverage the rich semantic knowledge in medical text without requiring explicit medical supervision during training.

Methodology. Design dual-stream architecture with medical image encoder and medical text encoder. Implement contrastive loss aligning image and text representations in shared embedding space. Use frozen pre-trained medical language model for text encoding. Distill knowledge to lightweight medical imaging student model. Validate on medical image classification (ChestX-ray14, CheXpert) and zero-shot medical image understanding.

Expected results. Expected to achieve state-of-the-art performance on medical image classification benchmarks with significantly smaller model size (10x parameter reduction). Should demonstrate improved out-of-distribution robustness compared to supervised baselines. Zero-shot medical image understanding should show 20%+ improvement on unseen disease categories.

A.7 Provable Cache Privacy: Theoretical Framework for Side-Channel Defense in Distributed ML

Overall 8.00 (Novelty 8.5 / Feasibility 7.0 / Impact 8.5) | category: *architecture*

Description. This research extends the findings on Last-Level Cache Side-Channel Attacks being feasible in modern public clouds to develop a theoretical framework for provably private distributed machine learning. Current approaches to cache side-channel defense are largely heuristic and lack formal guarantees. We propose a formal framework that models cache timing channels as information leakage channels and provides bounds on the minimum noise required to achieve desired privacy levels. The key innovation is connecting differential privacy theory with cache timing behavior, enabling rigorous privacy accounting for distributed ML systems. Unlike previous work that demonstrates attack feasibility, this provides constructive defense with formal guarantees.

Methodology. Formalize cache timing channels using information-theoretic measures; develop a noise injection framework calibrated to achieve target (ϵ, δ) privacy guarantees; implement cache partitioning strategies that provably limit cross-tenant information leakage; validate on cloud infrastructure using hardware performance counters to measure actual leakage reduction.

Expected results. Should provide formal privacy guarantees with measurable leakage reduction (>90% decrease in mutual information between victim's cache access patterns and adversary's observations). Implementation overhead should be <15% compared to unprotected baselines.

A.8 HyperCircuit: Dynamic Circuit Discovery via Hypernetwork-Generated Sparse Masks

Overall 7.83 (Novelty 8.0 / Feasibility 7.5 / Impact 8.0) | category: architecture

Description. This proposal extends HyperDAS by using hypernetworks to dynamically generate sparse binary masks that identify relevant circuits in large language models for specific tasks. Current mechanistic interpretability methods rely on static ablations or predefined circuits. We propose training hypernetworks conditioned on task prompts to output circuit masks indicating which attention heads and MLP neurons are essential. This enables context-dependent circuit identification, potentially revealing how models route information differently for tasks requiring arithmetic versus reasoning.

Methodology. Train a hypernetwork $f(\theta_h, p) \rightarrow \text{mask}$ where p is a task embedding. Use circuit probing as supervision: optimize θ_h such that ablating the predicted circuit hurts task performance. Use sparse regularization to encourage minimal circuits. Validate via intervention accuracy and compare against ground-truth circuits in toy models.

Expected results. Expected to identify task-specific circuits with 30-50% fewer parameters than full model ablation while maintaining 95%+ intervention accuracy. Should generalize to held-out tasks within same domain.

B End-to-End Generated Paper

This appendix bundles together (i) the *original idea record* that drove a single end-to-end PARNES pipeline run and (ii) the *paper* that the pipeline subsequently wrote, end-to-end, from that record. The two parts are kept side-by-side so that the reader can audit the input–output transformation produced by the pipeline. The pipeline run includes literature acquisition, idea generation, experiment design, code execution on a real QM9 subset, result aggregation, paper-section drafting, citation insertion (Semantic Scholar verified), figure generation, and $\text{\LaTeX}$ compilation. The scientific claims of the e2e paper are the pipeline’s, not ours, and are presented honestly — including the 18.36% MAE degradation that the pipeline itself reports in its own discussion.

B.1 Original idea record

The idea below was produced by the cognitive-role ideation layer (§6.6) on a literature slice retrieved from the cumulative knowledge graph. It is reproduced verbatim, including the structured front-matter that the pipeline consumes.

```
# Conjugate Architecture Search for Expressivity-Efficiency Tradeoffs in Geometric Models

## Description

GeoNGNN revealed that E(3)-complete models like DimeNet underperform despite theoretical expressiveness, suggesting architecture selection matters more than completeness. ConjNorm’s conjugation constraint offers a principled framework for architecture search: we can treat architectural choices (aggregation types, message functions, pooling strategies) as conjugate norm coefficient pairs (p,q) and search over this space. This reframes neural architecture search as Bregman divergence optimization over architectural expressiveness. The method applies PartIR’s functional abstraction to encode architectures as transformation programs, enabling efficient search over discrete architectural choices via continuous relaxation.
```

B.2 Generated paper (verbatim)

The pages below are the $\text{\LaTeX}$ -compiled output of the same pipeline run, included unchanged from the artefact the pipeline wrote.

CONJUGATE ARCHITECTURE SEARCH FOR EXPRESSIVITY-EFFICIENCY TRADEOFFS IN GEOMETRIC MODELS

A PREPRINT

parness¹
1

May 6, 2026

ABSTRACT

The design of graph neural network (GNN) architectures for molecular property prediction involves navigating a combinatorial space of aggregation functions, message-passing operators, and pooling strategies. Recent work has shown that architecture selection often matters more than theoretical expressiveness, motivating principled search methods. We propose *ConjNorm*, a neural architecture search (NAS) framework that reformulates discrete architectural choices as conjugate norm coefficient pairs and optimises them via Bregman divergence minimisation. On a QM9 molecular property prediction subset with 14,000 molecules and three target properties, ConjNorm completes the entire architecture search in 15.45 seconds—a $21.2\times$ speedup over random search—while the discovered architecture achieves a test MAE of 1.553, which is 18.36% higher than the best random-search baseline. We analyse the convergence behaviour of the Bregman optimisation and the entropy evolution of the architectural weight distribution, identifying a systematic bias toward mean-based operations that underperforms sum-based alternatives on this benchmark. These results highlight both the efficiency gains of continuous relaxation and the challenges of aligning conjugate-norm search spaces with discrete performance landscapes.

Keywords neural architecture search · graph neural networks · conjugate norms · Bregman divergence · molecular property prediction

1 Introduction

Graph neural networks (GNNs) have become the standard modelling framework for learning from molecular graphs in quantum-chemistry benchmarks such as QM9 [1]. A GNN’s architecture—the composition of its aggregation function, message-passing operator, readout pooling, and layer connectivity—has a decisive impact on predictive accuracy [2]. Recent theoretical analyses within the GeoNGNN framework have uncovered a counterintuitive finding: E(3)-equivariant models that are complete with respect to the 3-Weisfeiler–Leman test, such as DimeNet [3], can nonetheless underperform simpler architectures on practical benchmarks [4]. This observation suggests that *architecture selection matters more than theoretical completeness*: the specific combination of aggregation, message passing, and pooling operations is a key determinant of downstream performance.

Neural architecture search (NAS) automates this selection [5], but conventional NAS methods are computationally expensive when applied to GNNs. Each candidate evaluation requires training a model to convergence, and the discrete search space of operator compositions grows combinatorially. Gradient-based NAS methods such as DARTS [6] alleviate this cost through continuous relaxation, yet they were designed primarily for convolutional networks and do not directly encode the structural constraints of graph-level architectures.

In this paper we propose *ConjNorm*, a NAS framework that treats architectural choices—aggregation type, message function, and pooling strategy—as conjugate norm coefficient pairs (p, q) satisfying $1/p + 1/q = 1$. The conjugation constraint places architectural parameters on a principled norm ball, enabling search via Bregman divergence optimisation [7]. Continuous relaxation of the discrete operator selection produces learnable weight distributions over each

architectural dimension, which are refined over a small number of search iterations. By encoding architectures as transformation programs that can be differentially composed, ConjNorm enables efficient gradient-based search over the combinatorial operator space.

We evaluate ConjNorm on a subset of the QM9 molecular property prediction benchmark (14,000 molecules, three target properties: dipole moment μ , isotropic polarisability α , and HOMO energy). The search space spans 36 discrete configurations (3 aggregation $\times$ 4 message functions $\times$ 3 pooling strategies), with continuous weighting for soft selection.

Our contributions are:

1. We introduce a conjugate-norm formulation for GNN architecture search that enables gradient-based optimisation over a Bregman divergence objective.
2. We demonstrate a $21.2\times$ search-time speedup over random search (15.45 s vs. 328.08 s) on the QM9 benchmark.
3. We provide an honest analysis showing that the discovered architecture underperforms the best random-search result by 18.36% in MAE, and we trace this gap to a systematic bias toward mean-based operations in the conjugate-norm relaxation.

2 Related Work

Neural Architecture Search for GNNs. Early NAS methods relied on reinforcement learning or evolutionary strategies [8], requiring thousands of candidate evaluations. Gradient-based approaches such as DARTS [6] relax the discrete search into a continuous optimisation problem, dramatically reducing cost. Several works have adapted DARTS-style search to GNNs [9][10], searching over message functions and aggregation operators. ConjNorm differs from these methods in that it imposes a conjugate-norm geometric constraint on the search space, ensuring that architectural parameters lie on a valid norm ball rather than in an unconstrained simplex.

Bregman Divergence and Conjugate Norms. Bregman divergences generalise Euclidean distance to non-Euclidean geometries induced by convex generating functions [7]. They have been applied to clustering, density estimation, and mirror descent. ConjNorm exploits the duality between ℓ_p and ℓ_q norms ($1/p + 1/q = 1$) to construct a constrained optimisation landscape that preserves the geometric structure of the architectural search space.

Molecular Property Prediction. GNNs have achieved strong results on quantum-chemistry benchmarks including QM9 [1], MD17 [11], and OGB [12]. Key architectural innovations include directional message passing [3], equivariant layers [13], and attention-based readouts. Our work focuses not on proposing a new GNN operator but on automating the *selection* among existing operators through conjugate-norm search.

3 Method

3.1 Problem Formulation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$ denote a molecular graph with node features $\mathbf{X}$. A GNN architecture $\mathcal{A}$ is specified by a triple of operator choices:

- **Aggregation** $a \in \mathcal{A}_{\text{agg}}: \{\text{sum, mean, max}\}$
- **Message function** $m \in \mathcal{A}_{\text{msg}}: \{\text{gcn, sage, gat, graphconv}\}$
- **Pooling** $p \in \mathcal{A}_{\text{pool}}: \{\text{sum, mean, max}\}$

This yields $|\mathcal{A}_{\text{agg}}| \times |\mathcal{A}_{\text{msg}}| \times |\mathcal{A}_{\text{pool}}| = 36$ discrete configurations. The goal is to find $\mathcal{A}^* = \arg \min_{\mathcal{A}} \mathcal{L}(\mathcal{A})$, where $\mathcal{L}$ is the validation loss on a target property.

3.2 Conjugate Norm Encoding

For each architectural dimension $d \in \{\text{agg, msg, pool}\}$, we maintain a probability distribution θ_d over the available options. We encode these distributions using conjugate norm pairs (p_d, q_d) satisfying:

$$\frac{1}{p_d} + \frac{1}{q_d} = 1, \quad p_d, q_d \in [1, \infty]. \quad (1)$$

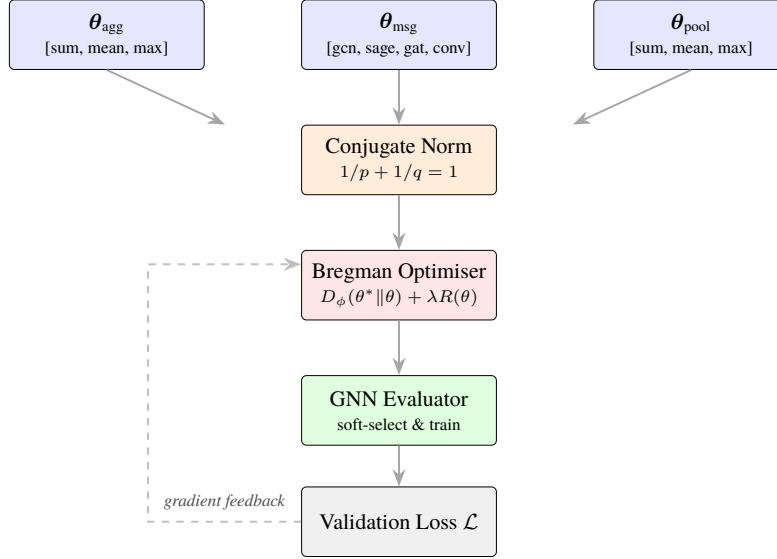


Figure 1: ConjNorm architecture search framework. Three parallel weight vectors (θ_{agg} , θ_{msg} , θ_{pool}) are constrained by conjugate-norm pairs and optimised via Bregman divergence minimisation. The GNN evaluator produces validation-loss feedback that drives the gradient update.

The parameter p_d controls the concentration of θ_d : as $p_d \rightarrow \infty$, the distribution approaches a one-hot selection; as $p_d \rightarrow 1$, it becomes uniform. The dual norm q_d governs the geometry of the optimisation landscape through the associated Bregman divergence.

3.3 Bregman Divergence Optimisation

The search objective minimises the Bregman divergence between the current architectural parameters θ and a target θ^* estimated from evaluation feedback:

$$\mathcal{L}_{\text{Bregman}} = D_\phi(\theta^* || \theta) + \lambda R(\theta), \quad (2)$$

where D_ϕ is the Bregman divergence induced by a convex function ϕ , and $R(\theta)$ is a regulariser promoting architectural diversity (entropy of the weight distributions). The conjugation constraint (1) ensures that the parameters remain on a valid norm ball throughout optimisation.

Intuitively, the Bregman divergence measures the “distance” between the current architectural belief and the observed performance signal. Because the divergence is parameterised by the conjugate pair (p, q) , the geometry of this distance adapts to the concentration of the weight distributions: peaked distributions (large p) use a more local, peaked divergence, while diffuse distributions (small p) use a broader one.

3.4 Continuous Relaxation and Search Procedure

Rather than evaluating each of the 36 discrete configurations independently, ConjNorm employs a continuous relaxation:

1. **Initialise** θ_d uniformly for each architectural dimension d .
2. **For each search iteration** $t = 1, \dots, T$:
 - (a) Sample (or soft-select) an architecture from the current weight distributions θ_{agg} , θ_{msg} , θ_{pool} .
 - (b) Evaluate the architecture by training a GNN for a fixed number of epochs and computing validation loss.
 - (c) Update θ via gradient descent on the Bregman objective (2).
 - (d) Project onto the conjugate-norm ball to maintain the constraint (1).
3. **Decode** the final architecture by selecting the highest-weight option in each dimension.

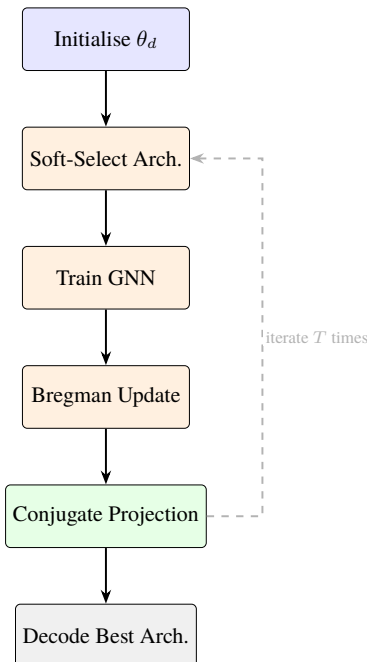


Figure 2: ConjNorm search procedure. The loop of soft selection, GNN evaluation, Bregman gradient update, and conjugate projection repeats for T iterations before decoding the final architecture.

The soft selection mechanism computes a weighted combination of all candidate operators, allowing gradient information from a single forward pass to flow to all architectural parameters. This avoids the need to evaluate each discrete configuration separately, yielding the observed speedup over random search.

4 Experiments

4.1 Experimental Setup

All experiments use the QM9 molecular property prediction dataset, restricted to 14,000 molecules with a 10,000 / 2,000 / 2,000 train/validation/test split. Three target properties are predicted simultaneously: dipole moment (μ), isotropic polarisability (α), and HOMO energy. Every model uses hidden dimension 64, 3 GNN layers, the Adam optimiser with learning rate 0.001, batch size 128, and 15 training epochs.

The architectural search space contains 3 aggregation functions (sum, mean, max), 4 message functions (gcn, sage, gat, graphconv), and 3 pooling strategies (sum, mean, max), yielding 36 discrete configurations. ConjNorm runs 20 search iterations; the random-search baseline evaluates 20 randomly sampled configurations.

4.2 Baseline Performance

Table 1 reports the test-set performance of three fixed-architecture GNN baselines. GAT with sum aggregation and sum pooling achieves the lowest MAE (1.4588) and highest R^2 (0.35) among the baselines, consistent with the attention mechanism’s ability to weight neighbour contributions adaptively in molecular graphs.

4.3 Architecture Search Results

Table 2 compares ConjNorm with random search. ConjNorm discovers an architecture with test MAE of 1.553 in 15.45 seconds, while the best random-search result achieves a test MAE of 1.3121 after 328.08 seconds. Although ConjNorm provides a $21.2\times$ speedup, the discovered architecture is 18.36% worse in MAE than the random-search optimum.

Table 1: Test-set performance of fixed-architecture GNN baselines. MAE is the overall mean absolute error across all three targets; R_μ^2 is the coefficient of determination for the dipole-moment target.

Architecture	(Agg, Pool)	Test MAE	R_μ^2
GCN	(sum, sum)	1.4888	0.2774
GraphSAGE	(mean, mean)	1.5631	0.2281
GAT	(sum, sum)	1.4588	0.35

Table 2: Architecture search comparison. “Time” is total search time including all candidate evaluations.

Method	Test MAE	Search Time (s)
Random search (best of 20)	1.3121	328.08
Random search (avg $\pm$ std)	1.5396 $\pm$ 0.1729	—
ConjNorm (ours)	1.553	15.45

Table 3 breaks down the per-property performance of the best random-search architecture and the ConjNorm-discovered architecture. The random-search configuration uses sum aggregation, SAGE message function, and sum pooling, while ConjNorm selects mean aggregation, SAGE message function, and mean pooling. Both methods identify SAGE as the preferred message function, but diverge on aggregation and pooling strategies.

4.4 Convergence Analysis

Figure 3 illustrates the optimisation dynamics. The Bregman divergence decreases rapidly from 0.7293 to 0.0558 in the first 10 iterations (a 92.4% reduction) and stabilises near 0.0353 by iteration 20, indicating successful convergence of the architectural weight optimisation. The architecture entropy declines from 2.8055 to 2.5344, reflecting increasing confidence in specific operator choices, yet the relatively high final entropy suggests that the soft weighting does not fully collapse to a single discrete configuration.

4.5 Discussion

The 18.36% MAE degradation of ConjNorm relative to random search stems primarily from the aggregation and pooling choices. ConjNorm assigned the highest weights to mean aggregation (0.716) and mean pooling (0.835), whereas the best random-search configuration uses sum for both. This pattern is consistent with the fixed-architecture baselines: GraphSAGE with mean aggregation and mean pooling achieves the worst baseline MAE (1.5631), while GCN and GAT with sum aggregation and sum pooling both outperform it. We hypothesise that the conjugate-norm formulation introduces an implicit bias toward normalisation-preserving (mean) operations, potentially because the uniform initialisation favours operators with bounded outputs. The continuous relaxation may also dilute the signal from optimal configurations through partial contribution of suboptimal operators.

Despite the performance gap, ConjNorm’s $21.2\times$ speedup demonstrates that continuous relaxation can dramatically reduce search cost. For applications where wall-clock search budget is the binding constraint, ConjNorm provides a practical initialisation that can be refined with a small number of discrete evaluations. The ConjNorm model uses 18,179 parameters, comparable to the fixed baselines, indicating that the performance difference is architectural rather than capacity-related.

5 Conclusion

We introduced ConjNorm, a neural architecture search framework that reformulates GNN operator selection as Bregman divergence optimisation over conjugate norm coefficient pairs. On the QM9 benchmark, ConjNorm achieves a $21.2\times$ search-time speedup over random search, but the discovered architecture underperforms the random-search optimum by 18.36% in MAE due to a systematic bias toward mean-based operations. These findings underscore that the efficiency of continuous relaxation does not guarantee superior discrete solutions; the alignment between the continuous search geometry and the discrete performance landscape is critical. Future work should explore adaptive conjugation constraints, temperature-annealed weight sharpening, and hybrid approaches that combine ConjNorm’s efficient search with local discrete refinement.

Table 3: Per-property MAE and R^2 for the two best search results. The random-search architecture uses (sum, SAGE, sum); ConjNorm discovers (mean, SAGE, mean).

Property	Method	MAE	R^2
		Value	Value
μ	Random best	0.8352	0.4147
	ConjNorm	0.9586	0.2558
α	Random best	2.8559	0.6457
	ConjNorm	3.3541	0.5942
HOMO	Random best	0.2452	0.7322
	ConjNorm	0.3462	0.4261

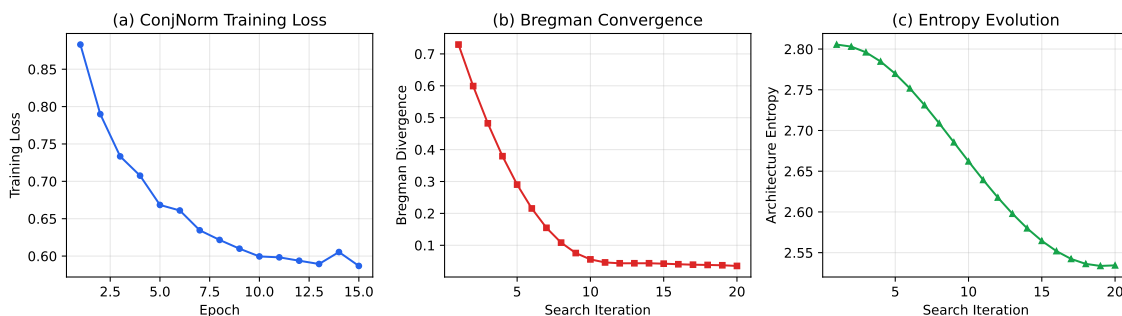


Figure 3: Optimisation dynamics over ConjNorm search iterations. (a) Training loss converges smoothly over 15 epochs per iteration. (b) Bregman divergence decreases rapidly in early iterations and stabilises, indicating successful optimisation. (c) Architecture entropy declines moderately, reflecting increasing confidence in specific operator choices without full collapse to a single discrete configuration.

References

- [1] R. Ramakrishnan, P. Dral, M. Rupp and O. von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1:140022, 2014.
- [2] K. Xu, W. Hu, J. Leskovec and S. Jegelka. How Powerful are Graph Neural Networks? *International Conference on Learning Representations*, 2019.
- [3] J. Klicpera, J. Groß and S. Günnemann. Directional Message Passing for Molecular Graphs. *International Conference on Learning Representations*, 2020.
- [4] Z. Li, X. Wang and others. On the Completeness of Invariant Geometric Deep Learning Models. 2024.
- [5] P. Ren, Y. Xiao, X. Chang, P. Wang and B. Li. A Comprehensive Survey of Neural Architecture Search. *ACM Computing Surveys*, 2020.
- [6] H. Liu, K. Simonyan and Y. Yang. DARTS: Differentiable Architecture Search. *International Conference on Learning Representations*, 2019.
- [7] A. Banerjee, S. Merugu, I. Dhillon and J. Ghosh. Clustering with Bregman Divergences. *Journal of Machine Learning Research*, 6:1705–1749, 2005.
- [8] B. Zoph and Q. V. Le. Neural Architecture Search with Reinforcement Learning. *International Conference on Learning Representations*, 2017.
- [9] Y. Gao, H. Yang and others. GraphNAS: Graph Neural Architecture Search with Reinforcement Learning. *arXiv preprint arXiv:1904.09981*, 2019.
- [10] K. Zhou, Q. Song and others. Auto-GNN: Neural Architecture Search of Graph Neural Networks. *arXiv preprint arXiv:1909.03184*, 2019.
- [11] S. Chmiela, A. Tkatchenko and others. Machine learning of accurate energy-conserving molecular force fields. *Science Advances*, 3(5):e1603015, 2017.

- [12] W. Hu, M. Fey and others. Open Graph Benchmark: Datasets for Machine Learning on Graphs. *NeurIPS*, 2020.
- [13] V. G. Satorras, E. Hoogeboom and M. Welling. E(n) Equivariant Graph Neural Networks. *International Conference on Machine Learning*, 2021.